

# A Petri Net Semantics for Acyclic Safe BPMN Choreographies

Gabriele Genovese<sup>1</sup> \*, Cinzia Di Giusto<sup>1</sup> , Ivan Lanese<sup>2</sup> \*\*, and  
Emilio Tuosto<sup>3</sup> 

<sup>1</sup> Université Côte d’Azur, CNRS, I3S, France

<sup>2</sup> Olas Team, University of Bologna & Inria - Université Côte d’Azur, Italy

<sup>3</sup> Gran Sasso Science Institute, Italy

**Abstract.** The specification of BPMN 2.0, a widely used notation for documenting and executing business processes, is written in natural language, hence it may be ambiguous. We formalise the semantics of acyclic and safe BPMN choreographies via Petri nets. We focus on the OR-join gateway, whose activation condition depends on how many paths have been chosen at the corresponding OR-split gateway, that is, by nature, a condition that cannot be observed locally. As in other works, we exploit the distinction between dead and alive paths to reduce this global condition to a local one, a technique called Dead Path Elimination. We prove that our semantics is equivalent to an existing ad-hoc approach from the literature. A key advantage of our approach is that it enables the use of well-established techniques and tools for the analysis of Petri nets.

**Keywords:** BPMN Choreography · Petri Net · Semantics · OR-gateway

## 1 Introduction

Business Process Model and Notation (BPMN) [25] is a widely adopted standard for the specification of business processes. This easy-to-learn graphical language features constructs for describing activities, control-flow decisions, synchronisation points, and interactions between participants. Besides process and collaboration diagrams, BPMN 2.0 introduced choreography diagrams [25, Chapter 11], which provide a more abstract description of the whole communicating system, focusing on the interactions among participants, without explicitly modelling their internal behaviour. The standard specification [25] is written in natural language, which may lead to ambiguities and interpretation issues.

This problem is particularly evident in the case of the *inclusive join gateway* (hereafter OR-join gateway). As described in the BPMN standard [25, page 352]:

---

\* This work has been supported by the French government, through the France 2030 investment plan managed by the Agence Nationale de la Recherche, as part of the “UCA DS4H” project, reference ANR-17-EURE-0004.

\*\* This work has been partially supported by French ANR project SmartCloud ANR-23-CE25-0012.

“Inclusive Gateways are used for modeling points of synchronization of a number of branches, not all of which are active, after which one or more alternative branches are chosen within a Choreography flow [...] and these are subsequently merged.”

The specification, however, does not clearly state the semantics of “subsequently merged”. Hence, the behaviour of a BPMN choreography varies across different execution engines. Also, it is often the case that implementations do not faithfully comply with the BPMN specification [7, Section 2.4]; for instance, Camunda [4] contradicts the specification, while jBPM [30] only supports OR-split gateways closed by a corresponding OR-join.

A common approach to the OR-join problem relies on the Dead Path Elimination (DPE) technique introduced in [19]. The key idea is to propagate information about inactive branches so that an OR-join can determine locally whether all relevant paths have been resolved (known as *wait for all active branches* policy). As discussed in Section 5, this technique has been declined in different ways [33,12,28]. In particular, building on [7], we propose a novel solution, which we spell out below.

**Structure of the paper and contributions.** After recalling basic elements of BPMN and an overview of the semantics proposed in [7] (cf. Section 2), we give our first contribution: an encoding into Petri nets of the formalisation of BPMN choreographies introduced in [6] (cf. Section 3). Our encoding captures the control flow induced by the gateways of the BPMN choreographies through the standard token-game semantics of Petri nets. Notably, the same gateways are used also in BPMN collaboration diagrams, the main difference being in tasks, which represent whole message exchanges, and that we model as atomic activities (see Section 6 for possible refinements). A key aspect of our encoding (cf. Definition 5) is the novel DPE mechanism: rather than superimposing DPE as an additional mechanism, we entangle it directly into the Petri net structure by adding “dead propagation” places and transitions. Through a simple static analysis of the syntactic representation of BPMN choreographies, we identify the portion of the Petri net capturing the “complementary” behaviour of choices. This enables the propagation of explicit signals on inactive paths so that join gateways can determine locally the resolution of each relevant path to compute the actual merge of the incoming branches.

Our main technical results show a precise semantic correspondence between the BPMN semantics and our encoding (cf. Theorems 1 to 3 in Section 4).

Our treatment of DPE yields a uniform Petri net semantics in which dead-path information becomes an intrinsic part of the model. The advantage of our approach is that it allows to re-use the vast theory and the tools based on Petri nets, our main motivation illustrated in Example 6. Indeed, our long-term goal is to provide a formal foundation for the analysis and verification of BPMN choreographies. This paper moves the first steps towards such goal paving the way to the adoption of the rich ecosystems of tool-supported modelling and verification techniques around Petri nets.

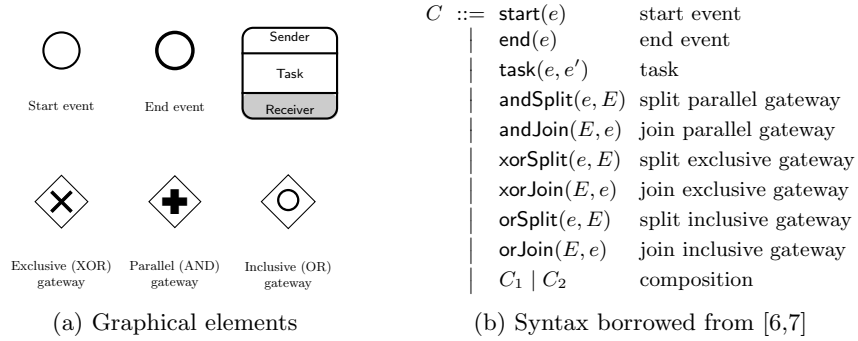


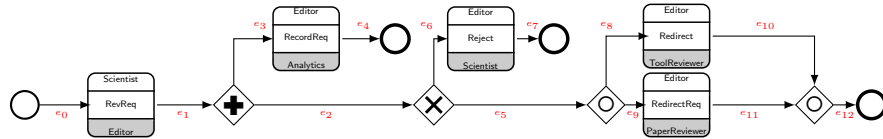
Fig. 1: Subset of considered BPMN elements

Finally, we consider related work (cf. Section 5 and the comparative Table 2) and discuss final remarks and future directions (cf. Section 6). In particular, we plan to extend our results to non-finitary BPMN models. In fact, our results hold on acyclic BPMN choreographies that, as in [7], we require to be *safe*, namely such that none of their terms is activated more than once.

## 2 Syntax and Semantics

In this paper, we consider only the BPMN elements depicted in Fig. 1a. The chosen elements capture the core control-flow behaviour and belong to the BPMN choreography part; however, apart from the task term, they can also be used for the corresponding elements in BPMN collaboration diagrams. Example 1 shows a BPMN choreography in the graphical notation of Fig. 1a.

*Example 1.* The BPMN diagram below specifies a reviewing process (red labels are instrumental for Example 2) where a review request from a scientist to an editor triggers two parallel activities (AND-split). One activity terminates once the editor logs the request for analytics purposes. In the other activity the editor evaluates whether the submission is out-of-scope or not (XOR-split).



A request is rejected if off-topic, otherwise the editor redirects it to reviewers, depending on the submission type (OR-split): research papers to a paper reviewer and tool papers to tool evaluators. A paper may fit both.  $\diamond$

To reason about BPMN choreographies, we follow the approach in [6,7], which represents BPMN choreographies with *BPMN terms* using the grammar in Fig. 1b (we slightly simplify the notation of [6,7]). Such terms include start

and end events, tasks, and three kinds of gateways: exclusive execution (XOR), parallel execution (AND), and inclusive execution (OR). Terms can be composed with the  $- \mid -$  operator. We call *basic* terms with no occurrences of the  $- \mid -$  operator. Connections between basic terms are defined by shared *edges*  $e$ , acting as references of *entry* and *exit points* of the control flow. We let  $E$  range over sets of edges. Each gateway comes in two variants: split and join. Split takes a single input and routes it towards a set of output edges; conversely, join synchronises multiple branches, having a set of input edges and a single output edge.

Briefly,  $\text{start}(e)$  represents a BPMN event that produces a token on outgoing edge  $e$ ; analogously,  $\text{end}(e)$  consumes a token from incoming edge  $e$ . A BPMN activity that consumes a token from  $e$  and produces one on  $e'$  is represented by the term  $\text{task}(e, e')$ . Gateway  $\text{andSplit}(e, E)$  consumes one token from  $e$  and produces tokens on all the edges in  $E$ . Its counterpart  $\text{andJoin}(E, e)$  requires tokens on all the edges in  $E$  to produce one token on  $e$ . Gateway  $\text{xorSplit}(e, E)$  consumes one token from  $e$  and produces one token on one (and only one) of the edges in  $E$ . Its counterpart  $\text{xorJoin}(E, e)$  is activated when at least one edge in  $E$  contains one token and produces one token on  $e$ . Finally,  $\text{orSplit}(e, E)$  consumes one token from  $e$  and produces a token on at least one edge in  $E$ . Conversely,  $\text{orJoin}(E, e)$  synchronises a subset of incoming edges in  $E$  producing a token on  $e$  when no more tokens are expected from the remaining incoming branches. This condition is what makes the treatment of the  $\text{orJoin}(E, e)$  gateway difficult: the activation of the rule depends on a condition on the “whole” choreography.

A basic term  $C'$  is *immediately reachable from*  $C$  if the exit points of  $C$  are not disjoint from the entry points of  $C'$ ; *syntactic reachability* is the transitive closure of immediate reachability.

*Example 2.* The choreography in Example 1 can be represented by the term:

$$\begin{aligned} C_{\text{ex}} = & \text{start}(e_0) \mid \text{task}(e_0, e_1) \\ & \mid \text{andSplit}(e_1, \{e_2, e_3\}) \\ & \mid \text{task}(e_3, e_4) \mid \text{end}(e_4) \\ & \mid \text{xorSplit}(e_2, \{e_5, e_6\}) \\ & \mid \text{task}(e_6, e_7) \mid \text{end}(e_7) \\ & \mid \text{orSplit}(e_5, \{e_8, e_9\}) \mid \text{task}(e_8, e_{10}) \mid \text{task}(e_9, e_{11}) \\ & \mid \text{orJoin}(\{e_{10}, e_{11}\}, e_{12}) \mid \text{end}(e_{12}) \end{aligned}$$

In  $C_{\text{ex}}$ ,  $\text{task}(e_0, e_1)$  is immediately reachable from  $\text{start}(e_0)$  and  $\text{end}(e_4)$  is syntactically reachable from  $\text{task}(e_0, e_1)$ , and hence from  $\text{start}(e_0)$ .  $\diamond$

For simplicity, we restrict our treatment to *well-formed* terms, namely those that are acyclic (in Section 6 we further discuss this assumption), have a unique occurrence of a  $\text{start}(e)$  term, at least one occurrence of an  $\text{end}(e)$  term, and every basic term is syntactically reachable from the  $\text{start}(e)$  term. Also, we do not consider multiple gateways (e.g., those with multiple entry or exit points) since they can always be replaced by an equivalent pair of separate split and join gateways [20,5].

We adopt the *local semantics* in [7, Fig. 11-13] whose pivotal notion of *marked process* is reflected by labelling edges with their *status* in the control flow. More

$$\begin{array}{c}
\frac{E' = \{e'.1\} \sqcup \text{setDead}(E)}{\text{xorSplit}(e.1, \{e'.\Sigma\} \sqcup E) \xrightarrow{(\mathbf{d}:\{e\} \sqcup E, 1:e')} \text{xorSplit}(e.\mathbf{d}, E')} \quad (\text{L-XorSplit-NC}) \\
\\
\frac{E' = \{e'.\mathbf{d}\} \sqcup \text{setDead}(E)}{\text{xorJoin}(\{e'.1\} \sqcup E, e.\Sigma) \xrightarrow{(\mathbf{d}:\{e'\} \sqcup E, 1:e)} \text{xorJoin}(E', e.1)} \quad (\text{L-XorJoin-NC}) \\
\\
\frac{E_1 \neq \emptyset \quad E = \text{setLive}(E_1) \sqcup \text{setDead}(E_2)}{\text{orSplit}(e.1, E_1 \sqcup E_2) \xrightarrow{(\mathbf{d}:\{e\} \sqcup E_2, 1:E_1)} \text{orSplit}(e.\mathbf{d}, E)} \quad (\text{L-OrSplit-NC}) \\
\\
\frac{\neg \text{isWait}(E) \quad \{e'.\Sigma \in E \mid \Sigma = 1\} \neq \emptyset}{\text{orJoin}(E, e.\Sigma) \xrightarrow{(\mathbf{d}:E, 1:e)} \text{orJoin}(\text{setDead}(E), e.1)} \quad (\text{L-OrJoin-NC}) \\
\\
\frac{\neg \text{isDead}(E), \quad \{e'.\Sigma \in E \mid \Sigma = 1\} = \emptyset}{\text{xorSplit}(e.\mathbf{d}, E) \xrightarrow{\mathbf{d}:E} \text{xorSplit}(e.\mathbf{d}, \text{setDead}(E))} \quad (\text{D-XorSplit}) \\
\\
\frac{\text{isDead}(E)}{\text{xorJoin}(E, e.\mathbf{w}) \xrightarrow{\mathbf{d}:e} \text{xorJoin}(E, e.\mathbf{d})} \quad (\text{D-XorJoin})
\end{array}$$

Fig. 2: Selected rules from [7]

precisely, the status of an edge  $e$  can be *live* ( $e.1$ ), *dead* ( $e.\mathbf{d}$ ), or *wait* ( $e.\mathbf{w}$ ) when  $e$  is respectively active, “deactivated”, or when the control flow has not reached  $e$  yet (cf. [7, Fig. 10]). We use  $\Sigma$  to range over statuses and  $\sqcup$  to denote disjoint union on sets of (possibly annotated) edges. We drop the status notation when we talk about the static structure of the BPMN term. The operational semantics is formalised as a labelled transition relation  $\xrightarrow{\alpha}$  over marked BPMN terms where labels  $\alpha$  are essentially partial maps from statuses to set of edges; e.g.,  $(\mathbf{d} : E, 1 : E')$  states that edges in  $E$  (resp.  $E'$ ) are dead (resp. live); curly brackets are omitted when  $E$  or  $E'$  are singletons and, likewise, round brackets are dropped when the domain of  $\alpha$  is a singleton. We recall some auxiliary notions from [7] to formalise the operational semantics. Given a set  $E$  of edges, functions  $\text{setLive}(E)$  and  $\text{setDead}(E)$  mark the edges in the set  $E$  as live and dead, respectively:

$$\text{setLive}(E) = \{e.1 \mid e \in E\} \quad \text{and} \quad \text{setDead}(E) = \{e.\mathbf{d} \mid e \in E\}$$

The predicate  $\text{isLive}(E)$  holds when every edge in  $E$  is live (and likewise for  $\text{isDead}(E)$ ), while  $\text{isWait}(E)$  holds when at least one edge in  $E$  is in the wait status. Predicate  $\text{isInit}(C)$  holds when all edges of  $C$  are in the wait state, capturing the initial configuration of the process before any step has been taken.

Here we focus on the rules of the semantics for XOR and OR gateways reported in Fig. 2; the full semantics can be found in Section A.

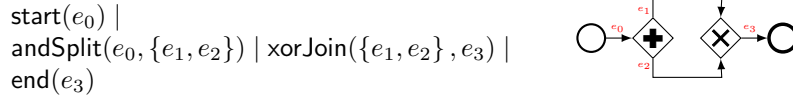
Rule **L-XorSplit-NC** models the execution of an XOR-split. When the incoming edge  $e.1$  is live, exactly one outgoing edge  $e'$  is non-deterministically selected and marked as live, while all remaining outgoing edges (in  $E$ ) are marked as dead. The label  $\alpha = (\mathbf{d} : \{e\} \sqcup E, 1 : e')$  highlights that edges in  $\{e\} \sqcup E$  are turned dead and edge  $e'$  is turned live. Dually, **L-XorJoin-NC**, for an XOR-join,

propagates a live status from an incoming edge  $e'.1$  to the outgoing edge  $e$ , and marks all the other incoming edges as dead ( $\text{setDead}(E)$ ). Rule L-OrSplit-NC models the execution of an OR-split. When the incoming edge  $e.1$  is live, the edges of a non-empty subset  $E_1$  of non-deterministically chosen edges of  $E$  are marked as live, while all remaining edges  $E_2 = E \setminus E_1$  are marked as dead. Rule L-OrJoin-NC describes the behaviour of the OR-join. The gateway is activated when none of the incoming edges are in the waiting state ( $\neg \text{isWait}(E)$ ), and at least one of them is live ( $\{e'.\Sigma \in E \mid \Sigma = 1\} \neq \emptyset$ ). The conditions are satisfied according to the DPE mechanism, which propagates dead statuses along the dead propagation branches and allows the OR-join to determine its enabling condition locally, as per the last two rules. Indeed, rules D-XorSplit and D-XorJoin ensure that once a branch status becomes dead, it is propagated through the process, enabling downstream gateways to correctly evaluate their activation conditions. In particular, each split gateway dead rule is activated when an incoming edge  $e$  is dead, all the outgoing edges are not dead ( $\neg \text{isDead}(E)$ ), and no outgoing edges are live. This ensures the propagation of the dead status through  $\text{setDead}(E)$ . Dually, each join gateway dead rule is activated when all incoming edges  $E$  are dead and the outgoing edge is in the wait status. Then, the rule can set  $e$  with the dead status. Dead status propagation for split (resp. join) gateways occurs in the same way also for  $\text{andSplit}(e, E)$  and  $\text{orSplit}(e, E)$  (resp.  $\text{andJoin}(E, e)$  and  $\text{orJoin}(E, e)$ ), whose rules are hence analogous (cf. Section A ). Hereafter,  $\mathbb{E}(C)$  is the set of edges of a BPMN term  $C$ .

Not all marked terms are meaningful: we restrict to those that can be reached by executing the term from configurations where all edges are in the wait state.

**Definition 1 (Reachable Marked Terms [7, Definition 2]).** *A marked term  $C$  is reachable if there exists a marked term  $C'$  such that  $\text{isInit}(C')$  holds and  $C' \twoheadrightarrow^* C$ , where  $\twoheadrightarrow^*$  is the reflexo-transitive closure of  $\twoheadrightarrow$ .*

Following [7], we formalise *safe* BPMN terms  $C$  as those for which in no run  $C \xrightarrow{\alpha_1} \dots \xrightarrow{\alpha_n}$  there are indexes  $i \neq j \in \{1, \dots, n\}$  such that  $\alpha_i = (\mathbf{d} : E_1, 1 : E_2)$  and  $\alpha_j = (\mathbf{d} : E'_1, 1 : E'_2)$  with  $E_2 \cap E'_2 \neq \emptyset$ . Thus, an edge can become live at most once during any run. For instance, the BPMN term (and its corresponding BPMN diagram) below is unsafe since



$e_3$  becomes “live” twice. Notice that, in our framework, safety is a decidable property since any well-formed term admits only finitely many reachable terms.

### 3 BPMN to Petri Net Encoding

We prove that the semantics presented in the previous section [7] can be faithfully simulated by Petri nets. As a result, BPMN models can be analysed using the large body of established Petri net theory and the numerous tools available for their verification. We start by recalling some notations and definitions:

**Definition 2 (Petri Net).** A Petri net  $N$  is a triple  $(P, T, F)$ , where  $P$  is a finite set of places,  $T$  a finite set of transitions, and  $F \subseteq (P \times T) \cup (T \times P)$  the flow relation. The preset and postset of a transition  $t \in T$  are defined as:  $\bullet t = \{p \in P \mid (p, t) \in F\}$  and  $t\bullet = \{p \in P \mid (t, p) \in F\}$ . A marking is a function  $M : P \rightarrow \mathbb{N}$  assigning a number of tokens to each place. A transition  $t$  is enabled under a marking  $M$  if  $M(p) \geq 1$  for all  $p \in \bullet t$ . The firing of a transition  $t$  enabled in a marking  $M$  produces the new marking

$$M'(p) = \begin{cases} M(p) - 1 & \text{if } p \in \bullet t \setminus t\bullet \\ M(p) + 1 & \text{if } p \in t\bullet \setminus \bullet t \\ M(p) & \text{otherwise} \end{cases}$$

We consider Petri nets equipped with an initial marking  $M_0$  and write  $(N, M) \rightarrow (N, M')$  when the firing<sup>4</sup> of a transition yields  $M'$  from  $M$ .

Our encoding of BPMN is compositional, hence we need to define the union operator on Petri nets.

**Definition 3 (Petri Net Union).** The union of two Petri nets  $N = (P, T, F)$  and  $N' = (P', T', F')$  is the Petri net  $N \oplus N' = (P \cup P', T \cup T', F \cup F')$ .

We informally introduce our approach for the encoding of BPMN terms into Petri nets, which is based on the DPE technique. While Corradini et al. [7] implement DPE at the level of an operational semantics by distinguishing between live and dead statuses, our approach embeds this distinction directly into the structure of the Petri-net encoding.

The key idea is to introduce a *dead propagation net*, namely a mirror copy of the relevant portion of the encoding in which each place and transition represents the dead counterpart of the corresponding element in the original net. The original net is used to propagate tokens representing a live status, while the mirror net propagates tokens representing a dead status. Consequently, gateways that may generate both live and dead outcomes, namely XOR-splits and OR-splits, are adapted to route tokens to the appropriate copy of the net.

The OR-join is connected to both the original and the dead propagation nets so that the completion of the synchronization can be determined locally: for each entry point, it receives a token either from the original net, corresponding to a live status, or from the dead propagation net, corresponding to a dead status.

Not all parts of the encoding require such a duplication. The dead propagation net is introduced only for the portion of the BPMN term that is syntactically reachable from an `xorSplit`( $e, E$ ) or an `orSplit`( $e, E$ ). Given a term  $C$ , we denote this set of edges by  $\phi(C)$ .

*Example 3.* The application of the function  $\phi$  to the BPMN term  $C_{\text{ex}}$  in Example 2 is the set  $\phi(C) = \{e_5, e_6, e_7, e_8, e_9, e_{10}, e_{11}, e_{12}\}$ .  $\diamond$

<sup>4</sup> Since transitions are immaterial for us, we prefer a reduction semantics style for the firing relation instead of the standard Petri net notation.

Our encoding exploits a preprocessing function that mirrors the part of the net necessary for DPE. Let  $\text{PN}(\cdot)$  be defined in Table 1 with a graphical representation of a simple instance of each case.

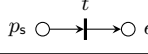
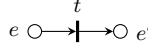
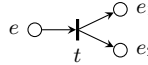
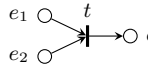
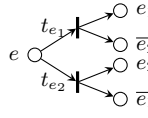
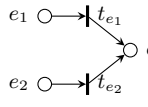
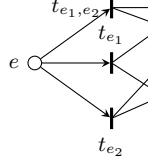
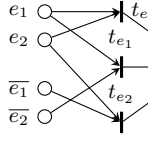
| Encoding (below $\mathcal{P}_\emptyset(X) = \{Y \subseteq X \mid Y \neq \emptyset\}$ )                                                                                                                                                                                                                                                                                       | Example                                                                              |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| $\text{PN}(\text{start}(e)) = (P, T, F)$ where<br>$P = \{e, p_s\}$ with $p_s$ fresh $T = \{t\}$ $F = \{(p_s, t), (t, e)\}$                                                                                                                                                                                                                                                   |    |
| $\text{PN}(\text{end}(e)) = (P, T, F)$ where<br>$P = \{e\}$ $T = \{t\}$ $F = \{(e, t)\}$                                                                                                                                                                                                                                                                                     |    |
| $\text{PN}(\text{task}(e, e')) = (P, T, F)$ where<br>$P = \{e, e'\}$ $T = \{t\}$ $F = \{(e, t), (t, e')\}$                                                                                                                                                                                                                                                                   |    |
| $\text{PN}(\text{andSplit}(e, E)) = (P, T, F)$ where<br>$P = \{e\} \cup E$ $T = \{t\}$ $F = \{(e, t)\} \cup \{(t, e')\}_{e' \in E}$                                                                                                                                                                                                                                          |    |
| $\text{PN}(\text{andJoin}(E, e)) = (P, T, F)$ where<br>$P = E \cup \{e\}$ $T = \{t\}$ $F = \{(e', t)\}_{e' \in E} \cup \{(t, e)\}$                                                                                                                                                                                                                                           |    |
| $\text{PN}(\text{xorSplit}(e, E)) = (P, T, F)$ where<br>$P = \{e\} \cup E \cup \{\bar{e} \mid e \in E\},$<br>$T = \{t_e \mid e \in E\},$<br>$F = \{(e, t_e), (t_e, e)\}_{e \in E} \cup \{(t_e, \bar{e'}) \mid e, e' \in E, e' \neq e\}$                                                                                                                                      |    |
| $\text{PN}(\text{xorJoin}(E, e)) = (P, T, F)$ where<br>$P = E \cup \{e\}$<br>$T = \{t_e \mid e \in E\}$<br>$F = \{(e, t_e), (t_e, e)\}_{e \in E}$                                                                                                                                                                                                                            |   |
| $\text{PN}(\text{orSplit}(e, E)) = (P, T, F)$ where<br>$P = \{e\} \cup E \cup \{\bar{e'} \mid e' \in E\}$<br>$T = \{t_S \mid S \in \mathcal{P}_\emptyset(E)\}$<br>$F = \{(e, t_S) \mid S \in \mathcal{P}_\emptyset(E)\} \cup \{(t_S, e') \mid S \in \mathcal{P}_\emptyset(E), e' \in S\} \cup \{(t_S, \bar{e'}) \mid S \in \mathcal{P}_\emptyset(E), e' \in E \setminus S\}$ |  |
| $\text{PN}(\text{orJoin}(E, e)) = (P, T, F)$ where<br>$P = \{e\} \cup E \cup \{\bar{e'} \mid e' \in E\}$<br>$T = \{t_S \mid S \in \mathcal{P}_\emptyset(E)\},$<br>$F = \{(t_S, e) \mid S \in \mathcal{P}_\emptyset(E)\} \cup \{(e', t_S) \mid S \in \mathcal{P}_\emptyset(E), e' \in S\} \cup \{(\bar{e'}, t_S) \mid S \in \mathcal{P}_\emptyset(E), e' \in E \setminus S\}$ |  |

Table 1: Petri net encoding

Intuitively, a live edge contributes a token on its corresponding place in the net while a dead edge contributes a token on its corresponding dead propagation place, provided that it is on a “frontier”, i.e., followed by an edge in the wait state. The set of such edges separates the portion of the BPMN process where propagation of dead notification already occurred from the one that has not yet been reached. Placing a token only at the frontier avoids flooding the dead propagation net with redundant tokens, and ensures that the OR-join can receive a dead token on each path corresponding to a non-activated original path.

The encoding is primarily inspired by the one in [11]. For start and end events, tasks, AND-split, and AND-join gateways the encoding is actually identical to the one in [11], barred for the end event, whose corresponding transition has in our case an empty postset. Let us focus on the encoding of the XOR-split and the OR gateways, for which our encoding differs from the one in [11].

**XOR split.** The encoding introduces one transition per exit point, all sharing the same input place. This models the non-deterministic choice: only one transition can fire, consuming the token in its preset. The fired transition, then, produces a token in the corresponding place, while simultaneously producing a token in each dead propagation place of the remaining exit point, representing the dead status that can be then propagated along the inactive branches.

**XOR join.** As in [11], the encoding introduces one transition per entry point, all sharing the same output place. This models a non-deterministic choice: each transition can fire independently when enabled, producing a token in the shared output place. The fact that exactly one entry point is active for a XOR-join is not forced here, but it follows from the safety condition: two active branches may result in two tokens in the shared place, violating safety. Note that one could use for the XOR-join an encoding which is the dual of the XOR-split. We have chosen this one for simplicity and to remain aligned, when possible, with [11]. The drawback is that some tokens which have no correspondence in BPMN semantics would remain in the dead propagation places, but as discussed in Example 5, we would have the same problem anyway, in more complex scenarios.

**OR split.** The encoding introduces a transition  $t_S$  for each non-empty subset  $S$  of  $E$ , representing the possible combinations of activated branches. All such transitions share the same input place; when a transition  $t_S$  fires, the token in its preset is consumed and a token is produced in each place of its postset, namely in each place  $e \in S$  and in each dead propagation place  $\bar{e}$  for  $e \in E \setminus S$ , representing the dead statuses. Hence, for every choice made by the OR-split, there exists exactly one transition whose firing produces the corresponding marking.

**OR join.** The encoding introduces one transition for each non-empty subset  $S$  of  $E$ , representing every allowed combination of active and inactive entry points. For a transition  $t_S$  to fire there must be a token in each  $e \in S$  and one in each  $\bar{e}$  for  $e \in E \setminus S$ . The firing of  $t_S$  consumes all such tokens and produces a single token in  $t_S \bullet = \{e\}$ . Therefore, the join is enabled only once every entry point has been resolved (*wait for all active branches* policy), either as live or dead, reducing the global activation condition of the OR-join to a local firing rule.

We discuss how the dead status can be propagated in our encoding which hinges on the *dead propagation net* (cf. Definition 4 below). Given a set of edges  $E$  in the original net, the dead propagation net will have edges  $\bar{E} = \{\bar{e} \mid e \in E\}$  corresponding to those of the original net; and likewise for some transitions. More precisely, the dead status will have a correspondence in the dead propagation net for every basic term except for XOR- and OR-gateways, whose dead propagation net differs. Given that the dead propagation rules D- $\star$  for all the split and join-gateways share the same premises and the same effects on the edges' status, those gateways behave the same in dead propagation net. Therefore, the encodings of

XOR-gateways and OR-gateways match those of the AND-gateways (w.r.t. the corresponding split and join rules).

**Definition 4 (Dead Propagation Net).** *The dead propagation net of a basic BPMN term  $C$  is the tuple  $\overline{PN}(C) = (\overline{P}, \overline{T}, \overline{F})$  where:*

- if  $C = \text{xorSplit}(e, E)$  or  $C = \text{orSplit}(e, E)$ , then  $\overline{P} = \{\overline{e}\} \cup \overline{E}$ ,  $\overline{T} = \{\overline{t}\}$ , and  $\overline{F} = \{(\overline{e}, \overline{t})\} \cup \{(\overline{t}, \overline{e'}) \mid e' \in E\}$  with  $\overline{t}$  fresh.
- if  $C = \text{xorJoin}(E, e)$  or  $C = \text{orJoin}(E, e)$ , then  $\overline{P} = \overline{E} \cup \{\overline{e}\}$ ,  $\overline{T} = \{\overline{t}\}$ , and  $\overline{F} = \{(\overline{e'}, \overline{t}) \mid e' \in E\} \cup \{(\overline{t}, \overline{e})\}$  with  $\overline{t}$  fresh;
- otherwise  $\overline{P} = \{\overline{p} \mid p \in P\}$ ,  $\overline{T} = \{\overline{t} \mid t \in T\}$ , and  $\overline{F} = \{(\overline{x}, \overline{y}) \mid (x, y) \in F\}$  with  $(P, T, F) = PN(C)$ .

The encoding function takes a reachable marked BPMN term and returns a marked Petri net.

**Definition 5 (Petri Nets Encoding).** *The encoding of a reachable marked (cf. Definition 1) BPMN term  $C$  is  $\llbracket C \rrbracket = (\widehat{C}, M)$  where*

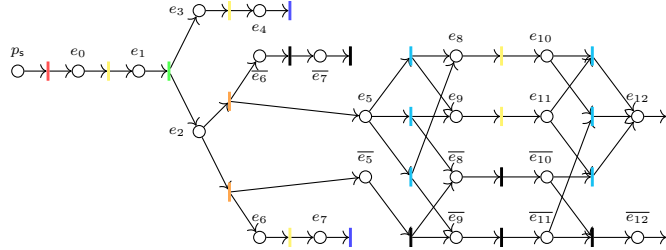
$$\widehat{C} = \begin{cases} PN(C) \oplus \overline{PN}(C) & \text{if } C \text{ is a basic term and } \mathbb{E}(C) \subseteq \phi(C) \\ \widehat{C}_1 \oplus \widehat{C}_2 & \text{if } C = C_1 \mid C_2 \\ PN(C) & \text{otherwise} \end{cases}$$

and marking  $M$  is such that  $M(p_s) = 1$  if  $\text{isInIt}(C)$  and

$$M(e) = \begin{cases} 1 & \text{if } e.1 \in \mathbb{E}(C) \\ 0 & \text{otherwise} \end{cases} \quad \text{and} \quad M(\overline{e}) = \begin{cases} 1 & \text{if there are } (\overline{e}, t), (t, \overline{e'}) \in F \\ & \text{with } e.d, e'.w \in \mathbb{E}(C) \\ 0 & \text{otherwise} \end{cases}$$

Note that, the encoding produces fresh and pairwise distinct transitions, hence no conflict is yield when composing encoded nets.

*Example 4.* The encoded version of the BPMN process in Example 3 is:



Transitions are colour-coded by component: red for the start event, blue for the end event, green for the AND-split, orange for the XOR-split, yellow for tasks, and cyan for the OR-split and OR-join. Black is used for every dead propagation transition, regardless of the component it belongs to.

The net starts with the fresh place  $p_s$ , introduced by the encoding of  $\text{start}(e_0)$ . From  $e_0$  to  $e_4$ , it contains the encoding of two tasks and an AND-split gateway.

Since these edges do not belong to the scope of any XOR-split or OR-split, no dead propagation places or transitions are required. The AND-split is encoded by a transition producing tokens on both  $e_2$  and  $e_3$ , thereby enabling the two branches in parallel. One of them leads to the XOR-split starting from  $e_2$ .

As shown in Example 3, all edges from  $e_5$  to  $e_{12}$  belong to  $\phi(C)$ . Consequently, every component in this region is encoded into both its standard and dead propagation representations. The effect of this construction is particularly evident in the encoding of  $\text{xorSplit}(e_2, \{e_5, e_6\})$ . When the transition selecting  $e_5$  fires, it produces a token on  $e_5$  and, at the same time, a token on  $\bar{e}_6$ . Thus, the choice of a branch immediately records the exclusion of the other one.

The next fragment contains the encoding of  $\text{orSplit}(e_5, \{e_8, e_9\})$  and the corresponding  $\text{orJoin}(\{e_{10}, e_{11}\}, e_{12})$ . Here, the powerset construction becomes explicit. The three cyan transitions connected to  $e_5$  represent all admissible branch selections: one transition activates both  $e_8$  and  $e_9$ , while the other two activate a single branch and simultaneously place a token in the dead propagation place of the discarded one. In this way, the net records both the selected and the excluded paths. Dually, the OR-join is encoded by a family of transitions that can fire only when each entry point has been classified as either live or dead. The synchronization condition is therefore enforced directly by the net structure.

The two tasks between the OR-split and the OR-join also belong to the scope of the preceding XOR-split and are equipped with their dead propagation counterparts. For example, the transition following  $\bar{e}_5$  propagates the dead status to both  $\bar{e}_8$  and  $\bar{e}_9$ . These tokens are eventually consumed by the dead propagation part of the OR-join, whose transitions collect tokens from  $\bar{e}_{10}$  and  $\bar{e}_{11}$ .

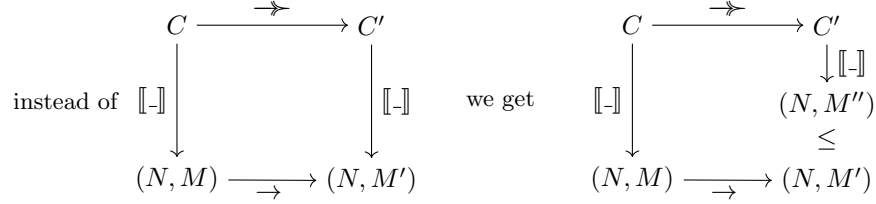
Finally, tokens are removed by the corresponding end-event encodings. Live tokens are consumed after  $e_4$  and  $e_{12}$ , whereas dead tokens are discarded by the dead propagation net corresponding encoding connected to  $\bar{e}_4$  and  $\bar{e}_{12}$ .  $\diamond$

*Remark 1.* We now discuss the size of the generated net. The number of places is linear in the number of edges of the translated BPMN term  $C$  as there is a place for each edge of  $C$  and possibly another one for the dead propagation net. The number of transitions in the translated Petri net is  $\mathcal{O}(|\mathbb{E}(C)| \cdot 2^{\text{MCD}(\mathbb{E}(C))})$  where  $|\mathbb{E}(C)|$  is the cardinality of  $\mathbb{E}(C)$  and  $\text{MCD}(\mathbb{E}(C))$  is its maximum connector degree [21] (i.e., the maximum number of exit and entry points of all the components in  $\mathbb{E}(C)$ ). The last term is actually exponential in  $\text{MCD}(\mathbb{E}(C))$  only for OR gateways, becoming linear for XOR gateways, and constant otherwise. This does not spoil the usability of the approach since the MCD of BPMN models is generally small, even more if only OR gateways are considered. Indeed, both the Average Connector Degree and the number of OR gateways have been shown to correlate negatively with the understandability of BPMN diagrams [21].

## 4 Soundness and Completeness

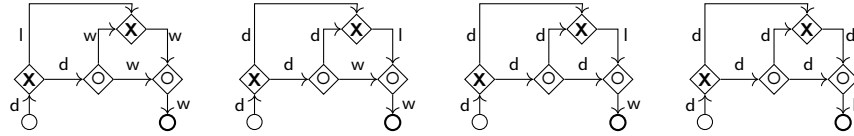
Having defined the encoding, we now establish that it faithfully preserves the operational semantics of marked BPMN terms. The soundness of the encoding

intuitively boils down to have a corresponding firing for every local reduction step in the BPMN semantics; one would like to have the commuting diagram on the left below, but our encoding ensures a weaker property represented by the commuting diagram on the right:

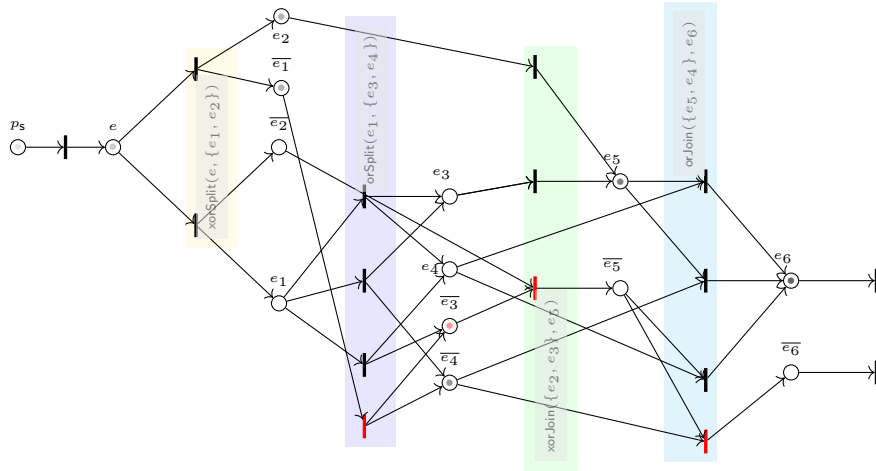


Essentially, a step  $C \xrightarrow{\alpha} C'$  can in general be matched in the encoding  $\llbracket C' \rrbracket = (N, M')$  “up-to” the presence of some tokens in the complement part of  $N$ . Such tokens may occur due, e.g., to the way dead status propagation interacts with the encoding of XOR-split and OR-join gateways.

*Example 5.* Consider the diagrams below illustrating a dead path propagation



The transition from the first to the second diagram propagates the live status  $l$  through the XOR-split. The propagation of status  $d$  through the OR-split in the second transition from the second to the third diagram is encoded as



where the red transitions are those belonging to the dead propagation net and the sequence of firings is represented by the strength of tokens' colour. The red token in place  $\bar{e}_4$  is stuck in the dead propagation net since the transition in its

postset is not enabled due to the fact that the XOR-split choose the branch on  $e_2$  (hence, the place  $\bar{e}_2$  is empty).  $\diamond$

Theorem 1 accounts for this small discrepancy in the correspondence by stating that the complement portion of the encoding may possibly have more tokens than the net encoding of the  $C'$  term.

**Theorem 1 (Soundness).** *Let  $C$  be a reachable marked BPMN term, and  $\llbracket C \rrbracket = (N, M)$  its encoding. If  $C \xrightarrow{\alpha} C'$  then there is a firing  $(N, M) \rightarrow (N, M')$  such that, assuming  $\llbracket C' \rrbracket = (N, M'')$ ,  $M'(e) = M''(e)$  for all  $e \in \mathbb{E}(C)$  and  $M''(\bar{e}) \leq M'(\bar{e})$  for all  $e \in \phi(C)$ .*

*Proof (sketch).* The proof is by induction on the height of the derivation of  $C \xrightarrow{\alpha} C'$ , with a case analysis on the last applied rule. Safety is exploited to ensure that in case of live token propagation, outgoing edges initially have wait status. The detailed case analysis can be found in Section C.  $\square$

For the completeness direction, we show that every firing step in the encoded net induces a BPMN reduction.

**Theorem 2 (Completeness).** *Let  $C$  be a reachable marked BPMN term, and  $\llbracket C \rrbracket = (N, M)$  its encoding. If  $(N, M) \rightarrow (N, M'')$ , then there exists a reachable marked BPMN term  $C'$  such that  $C \xrightarrow{\alpha} C'$  and  $(N, M') = \llbracket C' \rrbracket$ , for all  $e \in \mathbb{E}(C)$ ,  $M'(e) = M''(e)$  and for all  $e \in \phi(C)$ ,  $M''(\bar{e}) \geq M'(\bar{e})$ .*

*Proof (sketch).* The proof is by structural induction on the BPMN term  $C$ . In each case, we show that every transition of  $\hat{C}$  has a corresponding transition from  $C$ . The detailed case analysis can be found in Section D.

*Remark 2.* The presence of unconsumed tokens shown in Example 5 does not spoil the completeness of our encoding. In fact, these additional tokens “move”, if at all, in the dead propagation net portion of the encoding without affecting the marking corresponding to the other portion.

Theorems 1 and 2 establish a full operational correspondence between the BPMN local semantics and our Petri net encoding, confirming that the encoding is both sound and complete with respect to the local reduction relation  $\xrightarrow{\alpha}$ . Such a result can be lifted from transitions to computations.

**Theorem 3 (Correctness for Computations).** *Let  $C$  be a reachable marked BPMN term, and  $\llbracket C \rrbracket = (N, M)$  its encoding. We have  $C \xrightarrow{\alpha_1} \dots \xrightarrow{\alpha_n} C'$  iff there is a sequence of firings  $(N, M) \rightarrow (N, M_1) \rightarrow \dots \rightarrow (N, M_{m-1}) \rightarrow (N, M_m)$  for some  $m \geq n$  such that, assuming  $\llbracket C' \rrbracket = (N, M'')$ ,  $M_m(e) = M''(e)$  for all  $e \in \mathbb{E}(C)$  and  $M''(\bar{e}) \leq M_m(\bar{e})$  for all  $e \in \phi(C)$ .*

*Proof (sketch).* Both directions are by induction on the length of the computation, using, respectively, Theorem 1 and Theorem 2, for the inductive step. Soundness follows by monotonicity of firing. Completeness relies on Lemma 2 to show that spurious dead tokens do not trigger additional transitions.  $\square$

The next example uses a prototype [13] to demonstrate how to leverage an existing Petri net tool.

*Example 6.* The encoded Petri net in Example 4 is generated by our tool, which also outputs it in the input format for TINA [3], a tool which we adopt to verify safety. The analysis of TINA explored the state space of the choreography (565 markings and 1343 transition firings) without finding a violation of safety. By Theorem 3, the translations of reachable marked BPMN configurations preserve the number of tokens of the corresponding configurations. Therefore, the safety of the Petri net propagates to the original BPMN choreography.

As a sanity check, we also considered the unsafe choreography presented in Section 2. In this case, TINA detected a violating marking. The translations of the reachable configurations have the same tokens (since there are no dead propagation places in the Petri net). Hence, the violating marking corresponds to a BPMN configuration with the same token multiplicity. However, Theorem 3 does not provide a general preservation result for unsafe terms. Therefore, the detected violation is used as a counterexample that can guide an additional check of the corresponding BPMN execution.  $\diamond$

## 5 Related Work

Several formal semantics for BPMN have been proposed in the literature, relying on a variety of target languages and formal models that address, or not, the OR-join problem through different semantic mechanisms. Here we discuss only the works most closely related to our approach; Table 2 summarises them. The **Semantics** column reports the formalism used to define or encode the semantics, while **BP Lang.** indicates the variant of business process model considered, either a version of the standard or a related formalism. The **OR-join** column reports whether OR-gateways are supported. **L/G** distinguishes local approaches, where OR-join activation can be determined from the local state of the gateway, from global approaches, where it requires reasoning about the state of the whole process. Finally, the **DPE** column shows whether the DPE technique is used, and whether it requires to equip the semantics with specific mechanism for implementing the DPE technique (see discussion below). This comparison is meant to highlight the motivation behind our work: it is a first step towards a complete formalisation of the standard BPMN semantics.

The closest related work is Corradini et al. [7], which extends an earlier proposal [6] with OR-gateways. The operational semantics in [7] closely mirrors the standard, through a global and a local approach. The global approach imposes no restrictions. The local one requires safe nets as ours, and features cycles, which we do not support. A main drawback of the local approach is that it relies on additional data structures, i.e., waiting, live, or dead statuses.

From a second group of works [33,28,12], we draw inspiration for the DPE technique used in this paper. However, rather than treating DPE as an additional operational mechanism layered on top of the semantics, we integrate it

| Reference  | Semantics    | BP Lang. | OR-join | L/G    | DPE      |
|------------|--------------|----------|---------|--------|----------|
| This paper | Petri nets   | BPMN 2.0 | ✓       | Local  | Standard |
| [7] Global | LTS          | BPMN 2.0 | ✓       | Global | N/A      |
| [7] Local  | LTS          | BPMN 2.0 | ✓       | Local  | Adapted  |
| [33]       | LTS          | BPMN 2.0 | ✓       | Local  | Adapted  |
| [28]       | WF Graph     | BPMN 2.0 | ✓       | Local  | Adapted  |
| [12]       | WF Graph     | WF Lang. | ✓       | Local  | Adapted  |
| [29]       | CPN          | BPMN 2.0 | ✓       | Local  | -        |
| [8]        | CPN          | BPMN 2.0 | ✓       | Local  | -        |
| [23]       | CPN          | BPMN 2.0 | ✗       | N/A    | N/A      |
| [11]       | Petri nets   | BPMN 1.0 | ✗       | N/A    | N/A      |
| [31]       | Graph Rew.   | BPMN 2.0 | ✓       | Global | N/A      |
| [17]       | Graph Rew.   | BPMN 2.0 | ✓       | Global | N/A      |
| [14]       | FOL          | BPMN 2.0 | ✓       | Global | N/A      |
| [27]       | LOTOS NT     | BPMN 2.0 | ✓       | Global | N/A      |
| [18]       | Boolean nets | EPC      | ✓       | Local  | Adapted  |
| [22]       | Graph Rew.   | EPC      | ✓       | Local  | Adapted  |
| [1,16]     | Fixed point  | EPC      | ✓       | Global | N/A      |

Table 2: Synopsis of the comparison of related work

directly into the structure of the Petri net, through dedicated dead propagation places and transitions. This yields a uniform Petri net semantics in which dead-path information becomes an intrinsic part of the model. Note that [33,28], besides relying on an ad-hoc treatment of OR-gateway semantics, restrict attention, as we do, to acyclic nets: [33] marks each arc as “bottom”, “executed”, or “skipped”, with tokens that can be true or false, while [28] uses six kinds of tokens (none, live, dead, pending, previously live, and previously dead). The constructs studied in [12] feature loops and are conjectured to be encodable in BPMN-like languages; giving up to the standard Petri net semantics, the approach introduces three kinds of tokens (white, grey, black); moreover, it only considers “sound” nets, i.e., nets that are both safe and deadlock-free.

A different technique for relaxing the constraints on loops is to use higher-order tokens, as in coloured Petri nets (CPNs) [29,8]. However, [29,8] both require every split gateway to be paired with a corresponding join gateway, a constraint we do not impose (cf. Example 5). It is also worth mentioning [23,11], which target the most commonly used BPMN constructs but do not provide a formal, sound-and-complete treatment of OR-gateways. [23] uses CPNs but restricts to BPMN models without OR-gateways. Dijkman et al. [11], whose encoding we follow closely, instead use standard Petri nets and model OR-gateways as a combination of AND- and XOR-gateways, following [2]. This encoding, however, may admit behaviours that are not valid BPMN executions: in particular, the resulting net may allow runs in which none of the outgoing branches of an OR-split is selected, whereas BPMN requires at least one branch to be activated. Our encoding avoids this issue by explicitly representing every non-empty subset of selected branches. Although this yields a number of transitions exponential in

the number of outgoing branches, BPMN models typically have only a small number of alternatives per gateway, which keeps the approach practical in realistic scenarios (cf. Remark 1). Moreover, our encoding places tokens directly in their final destinations within a single transition firing, avoiding intermediate states and simplifying the proof of operational correspondence.

Less closely related are the works [31,17,14,27], addressing the OR-join problem through global reasoning rather than reducing OR-join activation to a purely local firing condition. Their main advantage is generality: since activation is not required to be locally decidable, these approaches can drop the syntactic restrictions, such as safety or gateway pairing, that local approaches, including ours, impose. Van Gorp and Dijkman [31] use graph transformations to formalise BPMN execution directly, whereas Kräuter et al. [17] rely on graph transformation systems as an analysis framework, translating BPMN models into a form amenable to existing graph-transformation tools. Houhou et al [14] give a first-order logic formalisation of BPMN collaborations, supporting parametric and time-aware verification. Poizat and Salaün [27] provide a semantics for BPMN choreographies including OR-gateways, by translation into LOTOS NT; their focus, however, is not the encoding itself but the study of choreography realisability (i.e., whether a choreography can be correctly implemented by a collaboration of independent participants), a point that we left for future investigations.

The last three rows of Table 2 concern Event-driven Process Chains (EPCs), a language closely related to BPMN that faces similar challenges in giving semantics to OR-join gateways. Langner et al. [18] translate EPCs into Boolean nets that propagate positive and negative tokens, in a manner similar to DPE, while Mendling and van der Aalst [22] separate state and context information to handle arbitrary EPCs. Both approaches, however, rely on richer formalisms such as coloured tokens with guards, or a two-dimensional marking, that go beyond standard Petri nets. The intricacies of formalising the OR-join’s semantics highlighted in [1] has been resolved in [16] using a fixed-point semantics. This non-local approach, different from ours, is sound for cyclic EPCs models.

The work in [9,24] and the studies of graphical choreographies via multiparty session types [10] or for healthcare process modelling [26] are not reported in Table 2 for the following reasons. The OR-join encodings in [9,24] can be unsound when multiple incoming branches carry a token, the transitions corresponding to different subsets may fire separately, duplicating tokens in the output place; instead of concerning a formalisation of BPMN semantics, the work in [10] focuses on communication correctness while the one in [26] on modelling.

## 6 Conclusion and Future Works

We presented a Petri net encoding of BPMN choreographies and proved its equivalence with the operational semantics proposed by Corradini et al. [7]. The main contribution lies in the treatment of OR-split and OR-join gateways, whose semantics represents one of the most challenging aspects of BPMN. The resulting encoding provides a rigorous foundation for the analysis of BPMN choreogra-

phies through well-established Petri net techniques. This opens the possibility of investigating, in future work, the detection of deadlocks and the verification of type-theoretic properties such as protocol conformance and realisability. This would require a straightforward extension of our syntax to include in task terms involved participants and exchanged message, as in [6]; this enables the definition of a *projection* operation and the identification of the realisability conditions.

This paper also establishes a well-grounded basis for extending the considered BPMN fragment towards additional BPMN collaboration constructs, incorporating, for example, send and receive events, event-based gateways, and message flows between pools and participants. These components are well documented in the literature, and their integration into our framework appears relatively direct.

A future direction is the extension of our approach to handle a broader class of BPMN terms. Firstly, we intend to support looping constructs. Extending the framework to cyclic models is particularly challenging. Our current encoding relies on the fact that dead-path information flows monotonically through the dead propagation net. In acyclic choreographies, once a dead token has been propagated along a branch, it can never influence the execution again. This property is crucial in the proofs of soundness and completeness, since extra tokens accumulating in dead propagation places remain harmless. In the presence of cycles, however, dead-path information may circulate indefinitely and interact with subsequent iterations of the process and it may incorrectly affect the enabling conditions of gateways reached in later iterations. Hence, handling cyclic choreographies requires to distinguish different traversals of a loop and to ensure that dead-path information is consumed or reset appropriately. Secondly, we plan to drop the safety assumption so to deal with BPMN terms where activities can be executed multiple times, e.g., as a consequence of an AND-split followed by a XOR-join. Notice that cycles and (un)safety are two sides of a same coin; in fact, loops allow the execution of activities due to the reuse of tokens while unsafety uses multiple (possibly concurrent) tokens to the same effect.

Another future research direction is to revise the treatment of XOR-join gateways so that they consume both live and dead information, symmetrically to the way XOR-split gateways generate it. Such a modification could prevent the accumulation of obsolete dead tokens while preserving the local nature of gateway activation. Investigating these alternatives and their correspondence with the BPMN standard is an interesting topic for future work.

Finally, our encoding is supported in a prototype [13] which we are extending with new features to integrate it with existing Petri net tools. Currently we support the standard PNML [32] format (used by TINA and other tools such as LoLA [34], and TAPAAL [15]). This enables the verification of several properties of the encoded choreography including, along the lines of the safety check in Example 6, deadlock freedom, boundedness, and reachability. Care will be needed to ensure that Theorem 3 allows one to port the results on BPMN terms.

*Acknowledgements.* We thank the anonymous reviewers for their useful comments and suggestions.

## References

1. van der Aalst, W.M., Desel, J., Kindler, E.: On the semantics of EPCs: A vicious circle. In: Nüttgens, M., Rump, F. (eds.) EPK. pp. 71–79 (2002)
2. van der Aalst, W.M., Ter Hofstede, A.H., Kiepuszewski, B., Barros, A.P.: Workflow patterns. *Distributed and parallel databases* **14**(1), 5–51 (2003)
3. Berthomieu, B., Vernadat, F.: Time Petri Nets Analysis with TINA. In: QEST. vol. 6, pp. 123–124 (2006)
4. Camunda services GmbH: Camunda v.7.14.0. <https://www.camunda.com> (2026)
5. Corradini, F., Ferrari, A., Fornari, F., Gnesi, S., Polini, A., Re, B., Spagnolo, G.O.: A guidelines framework for understandable BPMN models. *Data & Knowledge Engineering* **113**, 129–154 (2018)
6. Corradini, F., Morichetta, A., Polini, A., Re, B., Tiezzi, F.: Collaboration vs. choreography conformance in BPMN 2.0: From theory to practice. In: 2018 IEEE 22nd International Enterprise Distributed Object Computing Conference (EDOC). pp. 95–104. IEEE (2018)
7. Corradini, F., Muzi, C., Re, B., Rossi, L., Tiezzi, F.: BPMN 2.0 OR-Join Semantics: Global and local characterisation. *Information Systems* **105**, 101934 (2022)
8. Dechsupa, C., Vatanawood, W., Thongtak, A.: Transformation of the BPMN design model into a colored Petri net using the partitioning approach. *IEEE Access* **6**, 38421–38436 (2018)
9. Dehnert, J., Rittgen, P.: Relaxed soundness of business processes. In: International Conference on Advanced Information Systems Engineering. pp. 157–170. Springer (2001)
10. Deniérou, P.M., Yoshida, N.: Multiparty session types meet communicating automata. In: European Symposium on Programming. pp. 194–213. Springer (2012)
11. Dijkman, R.M., Dumas, M., Ouyang, C.: Semantics and analysis of business process models in BPMN. *Information and Software technology* **50**(12), 1281–1294 (2008)
12. Fahland, D., Völzer, H.: Dynamic skipping and blocking and dead path elimination for cyclic workflows. In: International Conference on Business Process Management. pp. 234–251. Springer (2016)
13. Genovese, G.: BPMNer: An encoder of BPMN choreographies to Petri nets. <https://bpmner.cappuccino.ovh/> (2026)
14. Houhou, S., Baarir, S., Poizat, P., Quéinnec, P., Kahloul, L.: A first-order logic verification framework for communication-parametric and time-aware BPMN collaborations. *Information systems* **104**, 101765 (2022)
15. Jensen, J.F., Nielsen, T., Oestergaard, L.K., Srba, J.: TAPAAL and reachability analysis of P/T nets. In: Transactions on Petri Nets and Other Models of Concurrency XI, pp. 307–318. Springer (2016)
16. Kindler, E.: On the semantics of EPCs: Resolving the vicious circle. *Data Knowl. Eng.* **56**(1), 23–40 (2006). <https://doi.org/10.1016/J.DATAK.2005.02.005>
17. Kräuter, T., Rutle, A., König, H., Lamo, Y.: Formalization and analysis of BPMN using graph transformation systems. In: International Conference on Graph Transformation. pp. 204–222. Springer (2023)
18. Langner, P., Schneider, C., Wehler, J.: Petri net based certification of event-driven process chains. In: International Conference on Application and Theory of Petri Nets. pp. 286–305. Springer (1998)
19. Leymann, F., Altenhuber, W.: Managing business processes as an information resource. *IBM systems journal* **33**(2), 326–348 (1994)

20. Mendling, J., Reijers, H.A., van der Aalst, W.M.: Seven process modeling guidelines (7PMG). *Information and software technology* **52**(2), 127–136 (2010)
21. Mendling, J., Reijers, H., Cardoso, J.: What Makes Process Models Understandable? In: *Business Process Management. Lecture Notes in Computer Science*, vol. 4714, pp. 48–63. Springer (2007)
22. Mendling, Jan and van der Aalst, Wil: Formalization and verification of EPCs with OR-joins based on state and context. In: *International Conference on Advanced Information Systems Engineering*, pp. 439–453. Springer (2007)
23. Najem, T., Perucci, A.: Mapping BPMN2 service choreographies to colored Petri nets. In: *International Conference on Software Engineering and Formal Methods*, pp. 85–100. Springer (2019)
24. Napolitano, A.: An Automatic, User-friendly Framework for Translating BPMN Diagrams to Petri Nets, <https://etd.adm.unipi.it/t/etd-03072025-112145/>
25. Object Management Group: Business Process Model and Notation (BPMN), Version 2.0. <https://www.omg.org/spec/BPMN/2.0/PDF> (Dec 2010), specification Version 2.0.2
26. Pal, S., Guanciale, R., Lanese, I., Tuosto, E., Clo, M.: Pomsets for Process Management: A Healthcare Case Study. In: *International Colloquium on Theoretical Aspects of Computing*, pp. 378–395. Springer (2025)
27. Poizat, P., Salaün, G.: Checking the realizability of BPMN 2.0 choreographies. In: *Proceedings of the 27th Annual ACM Symposium on Applied Computing*, pp. 1927–1934 (2012)
28. Prinz, T.M., Choi, Y., Vetterlein, A.: Progression: A lightweight BPMN engine simplifying the execution and monitoring of process models. In: *International Conference on Business Process Management*, pp. 291–307. Springer (2025)
29. Ramadan, M., Elmongui, H.G., Hassan, R.: BPMN formalisation using coloured Petri nets. In: *Proceedings of the 2nd GSTF annual international conference on software engineering & applications (SEA 2011)*, pp. 83–90 (2011)
30. Red Hat: jBPM v.7.48.0. <https://www.jbpm.org> (2026)
31. Van Gorp, P., Dijkman, R.: A visual token-based formalization of BPMN 2.0 based on in-place transformations. *Information and Software Technology* **55**(2), 365–394 (2013)
32. Weber, M., Kindler, E.: The Petri net markup language. In: *Petri Net Technology for Communication-Based Systems: Advances in Petri Nets*, pp. 124–144. Springer (2003)
33. Weidlich, M., Grosskopf, A., Barros, A.: Realising dead path elimination in BPMN. In: *2009 IEEE Conference on Commerce and Enterprise Computing*, pp. 345–352. IEEE (2009)
34. Wolf, K.: Petri net model checking with LoLA 2. In: *International Conference on Applications and Theory of Petri Nets and Concurrency*, pp. 351–362. Springer (2018)

## A BPMN semantics

The left-out operational rules of Corradini's semantics [7] are:

$$\begin{array}{c}
\text{start}(e.w) \xrightarrow{1:e} \text{start}(e.l) \quad (\text{L-Start-NC}) \\
\text{end}(e.l) \xrightarrow{d:e} \text{end}(e.d) \quad (\text{L-End-NC}) \\
\text{andSplit}(e.l, E) \xrightarrow{(d:e, 1:E)} \text{andSplit}(e.d, \text{setLive}(E)) \quad (\text{L-AndSplit-NC}) \\
\text{isLive}(E) \quad (\text{L-AndJoin-NC}) \\
\text{andJoin}(E, e.\Sigma) \xrightarrow{(d:e, 1:E)} \text{andJoin}(\text{setDead}(E), e.l) \\
\neg \text{isDead}(E) \quad \{e'.\Sigma \in E \mid \Sigma = 1\} = \emptyset \quad (\text{D-AndSplit}) \\
\text{andSplit}(e.d, E) \xrightarrow{d:E} \text{andSplit}(e.d, \text{setDead}(E)) \\
\text{isDead}(E) \quad (\text{D-AndJoin}) \\
\text{andJoin}(E, e.w) \xrightarrow{d:e} \text{andJoin}(E, e.d) \\
\neg \text{isDead}(E) \quad \{e'.\Sigma \in E \mid \Sigma = 1\} = \emptyset \quad (\text{D-OrSplit}) \\
\text{orSplit}(e.d, E) \xrightarrow{d:E} \text{orSplit}(e.d, \text{setDead}(E)) \\
\text{isDead}(E) \quad (\text{D-OrJoin}) \\
\text{orJoin}(E, e.w) \xrightarrow{d:e} \text{orJoin}(E, e.d) \\
M_1 \xrightarrow{\alpha} M'_1 \quad (\text{M-StatusUpd}_1) \\
M_1 \mid M_2 \xrightarrow{\alpha} M'_1 \mid M_2 \star \alpha \\
M_2 \xrightarrow{\alpha} M'_2 \quad (\text{M-StatusUpd}_2) \\
M_1 \mid M_2 \xrightarrow{\alpha} M_1 \star \alpha \mid M'_2
\end{array}$$

where the status update function  $\cdot \star \alpha$  updates edge statuses in a process according to the action  $\alpha$ , and is defined inductively over the process structure:

$$\begin{aligned}
\emptyset \star \alpha &= \emptyset & \text{andSplit}(e, E)(e.\Sigma, E) \star \alpha &= \text{andSplit}(e, E)(e.\Sigma \star \alpha, E \star \alpha) \\
(\{e.\Sigma\} \sqcup E) \star \alpha &= \{e.\Sigma \star \alpha\} \sqcup (E \star \alpha) & (M_1 \mid M_2) \star \alpha &= M_1 \star \alpha \mid M_2 \star \alpha \\
e.\Sigma \star (w : E_1, d : E_2, l : E_3) &= \begin{cases} e.w & \text{if } e \in E_1 \\ e.d & \text{if } e \in E_2 \\ e.l & \text{if } e \in E_3 \\ e.\Sigma & \text{otherwise} \end{cases}
\end{aligned}$$

Corradini et al. [7] focus on the control-flow behaviour of BPMN gateways and abstract away from atomic activities. Consequently, their semantics does not include rules for tasks. Since tasks are part of the BPMN fragment considered in this work, we extend the semantics with the two rules **L-Task-NC** and **D-Task**. The first one propagates a live token from the incoming edge to the outgoing edge, while the second propagates a dead token whenever no live token can arrive.

$$\text{task}(e.l, e'.\Sigma) \xrightarrow{(\mathbf{d}:e, \mathbf{l}:e')} \text{task}(e.d, e'.l) \quad (\text{L-Task-NC})$$

$$\text{task}(e.d, e'.w) \xrightarrow{\mathbf{d}:e'} \text{task}(e.d, e'.d) \quad (\text{D-Task})$$

## B Auxiliary Lemmata

**Definition 6 (Entry and Exit Points).** For a basic BPMN term  $C$ , let  $\text{in}(C)$  and  $\text{out}(C)$  denote its sets of entry and exit points, respectively, defined as:

$$\begin{aligned} \text{in}(\text{start}(e)) &= \emptyset & \text{out}(\text{start}(e)) &= \{e\}, \\ \text{in}(\text{end}(e)) &= \{e\} & \text{out}(\text{end}(e)) &= \emptyset, \\ \text{in}(\text{task}(e, e')) &= \{e\} & \text{out}(\text{task}(e, e')) &= \{e'\}, \\ \text{in}(\text{andSplit}(e, E)) &= \{e\} & \text{out}(\text{andSplit}(e, E)) &= E, \\ \text{in}(\text{andJoin}(E, e)) &= E & \text{out}(\text{andJoin}(E, e)) &= \{e\}, \\ \text{in}(\text{xorSplit}(e, E)) &= \{e\} & \text{out}(\text{xorSplit}(e, E)) &= E, \\ \text{in}(\text{xorJoin}(E, e)) &= E & \text{out}(\text{xorJoin}(E, e)) &= \{e\}, \\ \text{in}(\text{orSplit}(e, E)) &= \{e\} & \text{out}(\text{orSplit}(e, E)) &= E, \\ \text{in}(\text{orJoin}(E, e)) &= E & \text{out}(\text{orJoin}(E, e)) &= \{e\}. \end{aligned}$$

**Definition 7 (Structural Inclusion).** The structural inclusion relation  $\sqsubseteq$  on  $C \times C$  is the smallest reflexive and transitive relation such that, for any marked BPMN terms  $C$ ,  $C_1$ , and  $C_2$ , if  $C = C_1 \mid C_2$ , then  $C_1 \sqsubseteq C$  and  $C_2 \sqsubseteq C$ .

**Definition 8 (Syntactic Reachability).** Let  $C$  be a BPMN term. For basic BPMN terms  $C_1, C_2 \sqsubseteq C$ , we say that  $C_2$  is immediately reachable from  $C_1$  in  $C$ , written  $C_1 \rightsquigarrow_C^1 C_2$ , if

$$\text{out}(C_1) \cap \text{in}(C_2) \neq \emptyset.$$

We say that  $C_2$  is syntactically reachable from  $C_1$  in  $C$ , written  $C_1 \rightsquigarrow_C C_2$ , if  $\rightsquigarrow_C$  is the transitive closure of  $\rightsquigarrow_C^1$ .

**Definition 9 (Edge Path).** Let  $e, e'$  be edges occurring in  $C$ . A path from  $e$  to  $e'$  in  $C$  is a sequence of basic terms  $\pi = C_1, \dots, C_n$  ( $C_i \sqsubseteq C$ ) such that  $e \in \text{in}(C_1)$ ,  $e' \in \text{out}(C_n)$ , and

$$\text{out}(C_i) \cap \text{in}(C_{i+1}) \neq \emptyset \quad \text{for every } 1 \leq i < n.$$

We write  $\Pi_C(e, e')$  for the set of such paths, and  $|\pi| = n$  for the length of  $\pi$ . If  $e = e'$ , we let  $\Pi_C(e, e) = \{\varepsilon\}$  with  $|\varepsilon| = 0$ .

**Definition 10 (Edge Distance).** For edges  $e, e'$  occurring in  $C$ , define

$$\text{dist}_C(e, e') = \begin{cases} \min\{|\pi| \mid \pi \in \Pi_C(e, e')\} & \text{if } \Pi_C(e, e') \neq \emptyset, \\ \infty & \text{otherwise.} \end{cases}$$

We use the term  $\text{Join}(E, e)$  as  $\text{Join}(E, e) \in \{\text{andJoin}(E, e), \text{xorJoin}(E, e), \text{orJoin}(E, e)\}$  and, dually, the term  $\text{Split}(e, E)$  as  $\text{Split}(e, E) \in \{\text{andSplit}(e, E), \text{xorSplit}(e, E), \text{orSplit}(e, E)\}$ .

**Definition 11 (Candidate Split).** Let  $\text{Join}(E, e) \sqsubseteq C$ . A term  $\text{Split}(e', E') \sqsubseteq C$  is a split candidate for  $\text{Join}(E, e)$  if there is a injective map  $\beta : E \rightarrow \mathcal{P}_0(E')$  such that, for every  $e_i \in E$ , and for every edge  $e'' \in \beta(e_i)$   $\Pi_C(e'', e_i) \neq \emptyset$  (Definition 9), equivalently  $\text{dist}_C(e'', e_i) < \infty$  (Definition 10). We say  $\beta$  witnesses  $\text{Split}(e', E')$  as a split candidate for  $\text{Join}(E, e)$ , and we define its cost as

$$\text{cost}(\beta) = \sum_{e_i \in E} \sum_{e'' \in \beta(e_i)} \text{dist}_C(e'', e_i).$$

*Remark 3.* Such a  $\beta$  can be computed by a BFS over the BPMN term  $C$  from each  $e_j \in E'$ , collecting for every reachable  $e_i \in E$  a shortest witnessing path (and hence  $\text{dist}_C(e_j, e_i)$ ).

**Definition 12 (Minimal Candidate Split).** Let  $C_1 = \text{Join}(E, e) \sqsubseteq C$  and let  $C_2 = \text{Split}(e', E') \sqsubseteq C$  be a split candidate for  $\text{Join}(E, e)$ , witnessed by some injective  $\beta : E \rightarrow \mathcal{P}_0(E')$  (Definition 11). Define its cost as

$$\text{cost}(C_2) = \min \left\{ \text{cost}(\beta) \mid \beta \text{ witnesses } C_2 \text{ as a candidate split for } C_1 \right\}.$$

A candidate split  $C_2$  for  $C_1$  is a minimal candidate split if

$$\text{cost}(C_2) \leq \text{cost}(\text{Split}(e'', E''))$$

for every candidate split  $\text{Split}(e'', E'') \sqsubseteq C$  for  $\text{Join}(E, e)$ .

**Lemma 1 (Unique Minimal Upstream XOR-Split).** Let  $C$  be a well-formed, safe and reachable BPMN term, and let  $\text{xorJoin}(E, e) \sqsubseteq C$ . Then there is a unique minimal candidate split for  $\text{xorJoin}(E, e)$ , and it is a XOR-split.

*Proof.* By well-formedness every edge  $e_i \in E$  is syntactically reachable from the start term  $\text{start}(e_0)$ , and the paths from  $e_0$  to each edge of  $E$  must diverge at some split candidate gateway  $C' = \text{Split}(e', E') \sqsubseteq C$  with  $|E'| \geq 2$ , and there exist at least two distinct  $e_1 \neq e_2 \in E'$  each reaching respectively at least one, and different, elements of  $E$ .

We first show that the gateway  $C'$  must be a XOR-split. An AND-split cannot be such a gateway, since it activates all its outgoing branches simultaneously. Likewise, an OR-split cannot be such a gateway, since it may activate more than one outgoing branch. In either case, two or more branches of  $C'$  leading to  $\text{xorJoin}(E, e)$  could become live, and the subsequent execution of the  $\text{xorJoin}(E, e)$  would produce more than one token on  $e$ , contradicting safety. Hence, the branching decision that routes the flow into  $\text{xorJoin}(E, e)$  must be performed by an XOR-split.

We now show the minimality of the gateway  $C'$ , witnessed by some injective  $\beta'$ . Let  $\text{Split}(e'', E'') \sqsubseteq C$  be any split candidate, witnessed by some injective

map  $\beta^*$ . Since  $C$  is well-formed, every edge  $e''_i \in E''$  is reachable from  $e_0$ , so the concatenation of a path  $e_0 \rightsquigarrow e''_i$  with the path  $e''_i \rightsquigarrow e_i \in \Pi_C(e''_i, e_i)$  is a path in  $\Pi_C(e_0, e_i)$  for every  $e_i \in E$ ; every such path passes through  $e'$  that is precisely the point where the paths to  $E$  diverge in the minimal candidate split and  $e'$  occurs somewhere on it. If  $e'$  occurs in the segment  $e''_i \rightsquigarrow e_i$ , then

$$\text{dist}_C(e''_i, e_i) > \text{dist}_C(e', e_i)$$

since reaching  $e_i$  from  $e'$  requires at least one more step into some edge of  $E'$ .

If instead  $e'$  occurs strictly before  $e''_i$ , i.e. in the segment  $e_0 \rightsquigarrow e''_i$ , then  $\text{Split}(e'', E'')$  lies downstream of  $C'$ , entirely nested inside a single branch of  $E'$ . But  $C'$  has two branches reaching distinct elements of  $E$ , so there is some  $e_j \in E$ , reached only via the other branch, that  $\text{Split}(e'', E'')$  cannot reach, contradicting that  $\beta^*$  is defined on all of  $E$ . This case is therefore impossible. So, for every  $e_i \in E$  and every  $e''_i \in \beta^*(e_i)$ ,  $\text{dist}_C(e''_i, e_i) > \min_{e''' \in E'} \text{dist}_C(e''', e_i)$ , thus,

$$\text{cost}(\beta^*) = \sum_{e_i \in E} \sum_{e''_i \in \beta^*(e_i)} \text{dist}_C(e''_i, e_i) > \sum_{e_i \in E} \sum_{e''' \in E'} \text{dist}_C(e''', e_i) = \text{cost}(\beta').$$

Taking the minimum over all witnesses on both sides,  $\text{cost}(\text{Split}(e'', E'')) > \text{cost}(C')$  for every candidate  $\text{Split}(e'', E'') \neq C'$ . Hence  $C'$  is a minimal candidate split, and it is the *unique* one, since every other candidate has strictly greater cost. Since  $C'$  is an XOR-split, this concludes the proof.  $\square$

By Lemma 1, since  $C$  is well-structured, safe, and reachable, every XOR-join term has a unique upstream XOR-split that syntactically reaches it.

The uniqueness of this upstream gateway is essential for safety. Indeed, an  $\text{andSplit}(e, E)$  cannot be upstream of  $\text{xorJoin}(E, e)(E, e)$ , since it activates all its outgoing branches simultaneously, potentially causing multiple incoming branches of the  $\text{xorJoin}(E, e)$  to carry tokens at the same time. Similarly, an  $\text{orSplit}(e, E)$  activates a non-empty subset of its outgoing branches and may therefore activate multiple branches leading to the same  $\text{xorJoin}(E, e)$ . In either case, the  $\text{xorJoin}(E, e)$  could be activated more than once, causing multiple tokens to be produced on its output edge and violating safety. An  $\text{xorSplit}(e, E)$ , instead, activates exactly one outgoing branch and therefore preserves the mutual exclusion required by the  $\text{xorJoin}(E, e)$ .

**Lemma 2.** *Let  $C$  be a well-formed, safe and reachable marked BPMN term and  $\llbracket C \rrbracket = (N, M)$  its Petri net encoding with  $N = (P, T, F)$ . Let  $\text{xorJoin}(E, e) \sqsubseteq C$  be a sub-term of  $C$ , such that  $e$  has status live. Then there exists  $e' \in E$  such that for each firing  $(N, M[\overline{e'} \mapsto 1]) \rightarrow (N, M')$ , there is a firing  $(N, M) \rightarrow (N, M'')$ , such that for every edge  $e'' \in \mathbb{E}(C)$ ,  $M'(e'') = M''(e'')$ .*

*Proof (Proof Sketch).* By Lemma 1, there exists a unique  $\text{xorSplit}(e', E')$  that syntactically reaches  $\text{xorJoin}(E, e)$ . Moreover, since  $\text{xorJoin}(E, e)$  is a sub-term of  $C$  and the terms between this XOR-split and the  $\text{xorJoin}(E, e)$  are syntactically reachable, all edges connecting them belong to the scope of  $\phi$ . In particular,

$e' \in \phi(C)$  and  $E \subseteq \phi(C)$ . Since  $e$  is live, the  $\text{xorJoin}(E, e)$  has been triggered, and therefore the corresponding  $\text{xorSplit}(e', E')$  has already fired. By the encoding of  $\text{xorSplit}(e', E')$  (Table 1), firing produces a single live token in the selected branch and a dead token in each of the remaining  $|E| - 1$  dead propagation places. Regardless of the components between the  $\text{xorSplit}(e', E')$  and the  $\text{xorJoin}(E, e)$ , these dead tokens propagate through their dead propagation nets and eventually reach the incoming dead propagation places of the  $\text{xorJoin}(E, e)$ .

However, firing the dead propagation transition of the  $\text{xorJoin}(E, e)$  requires a dead token in *every* incoming dead propagation place. Let  $e' \in E$  be the branch through which  $e$  became live. This branch has no dead counterpart: its dead propagation place  $\neg e'$  never receives a token, since the branch was resolved as live, not dead. The dead propagation transition of the  $\text{xorJoin}(E, e)$  is therefore never enabled. Since the net contains no cycles, no token can return to re-trigger the  $\text{xorSplit}(e', E')$ . Hence,  $\neg e'$  remains permanently unmarked, while the dead tokens accumulated on the other dead propagation places remain permanently trapped. Consequently, every transition fired after  $e$  becomes live behaves identically whether or not these dead propagation places are marked, which proves the claim.  $\square$

The following lemma shows that updating  $C_1$  according to a step  $C_1 \xrightarrow{\alpha} C'_1$  can always be reflected onto the whole composed term  $C_1 \mid C_2$ : propagating the same status update  $\alpha$  to  $C_2$  via the  $\star$  operator does not break the correspondence between the BPMN semantics and its Petri net encoding. Concretely, we show that the marking obtained by firing the corresponding transition in the full net  $N$  still satisfies the two coherence conditions of Theorem 1 with respect to the encoding of the updated term  $C'_1 \mid C_2 \star \alpha$ . The proof proceeds by cases on how  $\alpha$  affects  $C_2$ ; in the case where the update reaches an  $\text{xorJoin}(E, e)$  whose dead propagation net may already carry tokens left over from a previously resolved sibling branch, we rely on Lemma 2 to argue that such tokens are harmless and do not affect the enabledness of subsequent transitions, hence do not threaten coherence.

**Lemma 3.** *Let  $C = C_1 \mid C_2$  be a reachable marked BPMN term,  $(N, M) = \llbracket C \rrbracket$ ,  $(N_1, M_1) = \llbracket C_1 \rrbracket$ , and  $(N_1, M'_1) = \llbracket C'_1 \rrbracket$ . If  $C_1 \xrightarrow{\alpha} C'_1$  and  $(N_1, M_1) \rightarrow (N_1, M'_1)$  with  $M'_1(e) = M_1(e)$  for all  $e \in \mathbb{E}(C_1)$  and  $M'_1(\bar{e}) \geq M_1(\bar{e})$  for all  $e \in \phi(C_1)$ , then there exists a firing  $(N, M) \rightarrow (N, M'')$  with  $M''(e) = M'_1(e)$  for all  $e \in \mathbb{E}(C)$  and  $M''(\bar{e}) \geq M'_1(\bar{e})$  for all  $e \in \phi(C)$ , where  $(N, M') = \llbracket C'_1 \mid C_2 \star \alpha \rrbracket$ .*

*Proof.* We proceed by structural induction on  $C = C_1 \mid C_2$ . We show the most interesting cases; the others are similar. In each case, the firing in  $(N, M)$  is the same transition that fires in  $(N_1, M_1)$ , since  $N_1$  is a subnet of  $N$ . The place update is automatically reflected in  $N_2$  for shared places, which corresponds to the  $\star$  operator at the BPMN level.

**Start:** Let  $C_1 = \text{start}(e.w)$  and  $C_1 \xrightarrow{\alpha} C'_1$  via rule L-Start-NC, with  $\alpha = 1 : e$ . By definition,  $M_1(p_s) = 1$  and  $M_1(e) = 0$ . In  $(N, M)$ , the same holds since  $N_1 \subseteq N$ : transition  $t_s$  fires, consuming the token from  $p_s$  and producing one

in  $e$ , yielding  $M''$  with  $M''(p_s) = 0$  and  $M''(e) = 1$ . Since  $C_2 \star \alpha = C_2 \star 1 : e = C'_2$ , by definition of  $\star$ ,  $M'(e) = 1$  in  $\llbracket C'_1 \mid C'_2 \rrbracket$ . The first condition holds:  $M''(e) = 1 = M'(e)$  for all  $e \in \mathbb{E}(C)$ . The second condition holds because  $\text{start}(e)$  introduces no places in the dead propagation net, hence  $M''(\bar{e}) = 0 = M'(\bar{e})$ .

**End:** Let  $C_1 = \text{end}(e.1)$  and  $C_1 \xrightarrow{\alpha} C'_1$  via rule L-End-NC, with  $\alpha = \mathbf{d} : e$ . By definition,  $M_1(e) = 1$ . Transition  $t_e$  fires in  $(N, M)$ , consuming the token from  $e$ , yielding  $M''$  with  $M''(e) = 0$ . Since  $C_2 \star \alpha = C_2 \star \mathbf{d} : e = C'_2$ , by definition of  $\star$ ,  $M'(e) = 0$  in  $\llbracket C'_1 \mid C'_2 \rrbracket$ . The first condition holds:  $M''(e) = 0 = M'(e)$  for all  $e \in \mathbb{E}(C)$ . The second condition holds trivially:  $e$  is live in  $C_1$ , so by safety  $M(\bar{e}) = 0$ ; firing  $t_e$  produces no token in the dead propagation net, hence  $M''(\bar{e}) = 0 = M'(\bar{e})$ .

**Task:** Let  $C_1 = \text{task}(e.1, e')$  and  $C_1 \xrightarrow{\alpha} C'_1$  via rule L-Task-NC, with  $\alpha = \mathbf{d} : e, 1 : e'$ . By definition,  $M_1(e) = 1$  and  $M_1(e') = 0$ . Transition  $t$  fires in  $(N, M)$ , consuming the token from  $e$  and producing one in  $e'$ , yielding  $M''$  with  $M''(e) = 0$  and  $M''(e') = 1$ . Since  $C_2 \star \alpha = C_2 \star (\mathbf{d} : e, 1 : e') = C'_2$ , by definition of  $\star$ ,  $M'(e) = 0$  and  $M'(e') = 1$  in  $\llbracket C'_1 \mid C'_2 \rrbracket$ . The first condition holds:  $M''(e) = 0 = M'(e)$  and  $M''(e') = 1 = M'(e')$ . The second condition holds trivially:  $e$  is live in  $C_1$  and  $e'$  is live in  $C'_1$ , so by safety  $M(\bar{e}) = 0$  and  $M'(\bar{e}') = 0$ ; firing  $t$  produces no token in the dead propagation net, hence  $M''(\bar{e}) = 0 = M'(\bar{e})$  and  $M''(\bar{e}') = 0 = M'(\bar{e}')$ . The case  $\Sigma = \mathbf{d}$  uses rule D-Task, and is analogous.

**OR-split:** Let  $C_1 = \text{orSplit}(e.1, E)$  and  $C_1 \xrightarrow{\alpha} C'_1$  via rule L-OrSplit-NC, with  $E_1 \neq \emptyset$ ,  $E_2 = E \setminus E_1$ , and  $\alpha = \mathbf{d} : \{e\} \cup E_2$ ,  $1 : E_1$ . By definition,  $M_1(e) = 1$  and  $M_1(e') = 0$  for all  $e' \in E$ . Transition  $t_{E_1}$  fires in  $(N, M)$ , consuming the token from  $e$  and producing one token in each  $e' \in E_1$  and one token in each  $e'' \in E_2$ , yielding  $M''$  with  $M''(e) = 0$ ,  $M''(e') = 1$  for all  $e' \in E_1$ , and  $M''(e'') = 1$  for all  $e'' \in E_2$ . Since  $C_2 \star \alpha = C'_2$ , by definition of  $\star$ ,  $M'$  agrees with  $M''$  on all shared edges. The first condition holds:  $M''(e''') = M'(e''')$  for all  $e''' \in \mathbb{E}(C)$ . For the second condition, each  $e'' \in E_2$  may or may not be a frontier dotted edge, so  $M'(\bar{e}'') \in \{0, 1\}$ ; since  $M''(\bar{e}'') = 1$ , we have  $M''(\bar{e}'') \geq M'(\bar{e}'')$ . For  $e' \in E_1$ , by safety  $M'(\bar{e}') = 0$  and  $M''(\bar{e}') = 0$ , so equality holds trivially. Together, both conditions are satisfied, concluding the case.

**OR-join:** Let  $C_1 = \text{orJoin}(E, e)$  and  $C_1 \xrightarrow{\alpha} C'_1$  via rule L-OrJoin-NC, with  $E_1 \neq \emptyset$ ,  $E_2 = E \setminus E_1$ , and  $\alpha = 1 : e$ . By definition,  $M_1(e') = 1$  for all  $e' \in E_1$ ,  $M_1(\bar{e}'') = 1$  for all  $e'' \in E_2$ , and  $M_1(e) = 0$ . Transition  $t_{E_1}$  fires in  $(N, M)$ , consuming all tokens from  $E_1$  and  $\bar{E}_2$  and producing one token in  $e$ , yielding  $M''$  with  $M''(e) = 1$  and  $M''(e') = 0$ ,  $M''(\bar{e}'') = 0$  for all  $e' \in E_1$ ,  $e'' \in E_2$ . Since  $C_2 \star \alpha = C'_2$ , by definition of  $\star$ ,  $M'(e) = 1$  in  $\llbracket C'_1 \mid C'_2 \rrbracket$ . The first condition holds:  $M''(e''') = M'(e''')$  for all  $e''' \in \mathbb{E}(C)$ . For the second condition,  $e$  may or may not be a frontier dotted edge, so  $M'(\bar{e}) \in \{0, 1\}$ ; since  $M''(\bar{e}) = 0$ , we need  $M''(\bar{e}) \geq M'(\bar{e})$ , which holds since  $e$  is live in  $C'_1$  and by safety  $M'(\bar{e}) = 0$ . Together, both conditions are satisfied, concluding the case.

**M-StatusUpd<sub>1</sub>**: Let  $C_1 = C_1^A \mid C_1^B$ , so  $C = C_1^A \mid C_1^B \mid C_2$ . Assume  $C_1^A \xrightarrow{\alpha} C_1^{A'}$  via rule **M-StatusUpd<sub>1</sub>**, with action  $\alpha$ ; the other case is symmetric. Then  $C \xrightarrow{\alpha} C_1^{A'} \mid C_1^B \star \alpha \mid C_2 \star \alpha = C_1^{A'} \mid C_1^B \mid C_2 \star \alpha$ . By inductive hypothesis applied to  $C_1^A$  and  $C_1^B \mid C_2$ , the firing  $(N_1^A, M_1^A) \rightarrow (N_1^A, M_1^{A'})$  satisfies the two conditions with respect to  $\llbracket C_1^{A'} \rrbracket$ . If this update reaches an **xorJoin**( $E, e$ ) whose dead propagation net already carries tokens left over from a previously resolved sibling branch, Lemma 2 ensures such tokens are harmless, since they do not affect the enabledness of any subsequent transition, and hence do not threaten coherence. By Lemma 3 there exists a firing  $(N, M) \rightarrow (N, M'')$  such that  $M''(e) = M'(e)$  for all  $e \in \mathbb{E}(C)$  and  $M''(\bar{e}) \geq M'(\bar{e})$  for all  $e \in \phi(C)$ , where  $(N, M') = \llbracket C_1^{A'} \mid C_1^B \mid C_2 \star \alpha \rrbracket$ , concluding the case.  $\square$

## C Soundness

**Theorem 1 (Soundness).** *Let  $C$  be a reachable marked BPMN term, and  $\llbracket C \rrbracket = (N, M)$  its encoding. If  $C \xrightarrow{\alpha} C'$  then there is a firing  $(N, M) \rightarrow (N, M')$  such that, assuming  $\llbracket C' \rrbracket = (N, M'')$ ,  $M'(e) = M''(e)$  for all  $e \in \mathbb{E}(C)$  and  $M''(\bar{e}) \leq M'(\bar{e})$  for all  $e \in \phi(C)$ .*

*Proof.* Recall that we only consider well-formed, safe and reachable marked BPMN terms. Safety is exploited to ensure that in case of live token propagation, outgoing edges initially have wait status. By induction on the height of the derivation of  $C \xrightarrow{\alpha} C'$ , with a case analysis on the applied rules.

**L-Start-NC:** Let  $\text{PN}(\text{start}(e)) = (P, T, F)$  with  $P = \{p_s, e\}$ ,  $T = \{t_s\}$ , and  $F = \{(p_s, t_s), (t_s, e)\}$ . Recall the **L-Start-NC** rule:

$$\frac{}{\text{start}(e.w) \xrightarrow{\alpha} \text{start}(e.l)}$$

where  $C = \text{start}(e.w)$ ,  $C' = \text{start}(e.l)$ ,  $\text{PN}(C) = (N, M)$ , and  $\text{PN}(C') = (N, M')$ . By definition,  $M(p_s) = 1$  and  $M(e) = 0$ , while  $M'(p_s) = 0$  and  $M'(e) = 1$ . The transition  $t_s$  has  $\bullet t_s = \{p_s\}$  and  $t_s \bullet = \{e\}$ ; therefore it is enabled under  $M$ . Firing  $t_s$  produces one token in  $e$ , yielding a marking  $M''$  such that  $M''(p_s) = 0$  and  $M''(e) = 1$ . The first condition holds since  $M''(e) = M'(e)$ . The second condition holds vacuously: **start**( $e$ ) introduces no dead propagation places, since its output edge cannot be a dotted edge by definition of  $\phi$ .

**L-End-NC:** Let  $\text{PN}(\text{end}(e)) = (P, T, F)$  with  $P = \{e\}$ ,  $T = \{t_e\}$ , and  $F = \{(e, t_e)\}$ . Recall the **L-End-NC** rule:

$$\frac{}{\text{end}(e.l) \xrightarrow{\alpha} \text{end}(e.d)}$$

where  $C = \text{end}(e.l)$ ,  $C' = \text{end}(e.d)$ ,  $\text{PN}(C) = (N, M)$ , and  $\text{PN}(C') = (N, M')$ . By definition,  $M(e) = 1$  and  $M'(e) = 0$ . The transition  $t_e$  has  $\bullet t_e = \{e\}$  and  $t_e \bullet = \emptyset$ ; therefore it is enabled under  $M$ . Firing  $t_e$  removes

one token from  $e$ , yielding a marking  $M''$  such that  $M''(e) = 0$ . The first condition holds since  $M''(e) = M'(e)$ . The second condition holds trivially:  $e$  is live in  $C$ , so by safety  $M(\bar{e}) = 0$ ; firing  $t_e$  produces no token on the dead propagation net, hence  $M''(\bar{e}) = 0 = M'(\bar{e})$ .

**L-Task-NC:** Let  $\text{PN}(\text{task}(e, e')) = (P, T, F)$  with  $P = \{e, e'\}$ ,  $T = \{t\}$ , and  $F = \{(e, t), (t, e')\}$ . Recall the L-Task-NC rule:

$$\frac{}{\text{task}(e.1, e'.\Sigma) \xrightarrow{\alpha} \text{task}(e.d, e'.1)}$$

where  $C = \text{task}(e.1, e'.\Sigma)$ ,  $C' = \text{task}(e.d, e'.1)$ ,  $\text{PN}(C) = (N, M)$ , and  $\text{PN}(C') = (N, M')$ . For safety,  $e'.\Sigma$  must be wait. By definition,  $M(e) = 1$ ,  $M(e') = 0$ , while  $M'(e) = 0$  and  $M'(e') = 1$ . The transition  $t$  has  $\bullet t = \{e\}$  and  $t\bullet = \{e'\}$ ; hence it is enabled under  $M$ . Firing  $t$  removes one token from  $e$  and adds one token in  $e'$ , yielding a marking  $M''$  such that  $M''(e) = 0$  and  $M''(e') = 1$ . The first condition holds since  $M''(e) = M'(e)$  and  $M''(e') = M'(e')$ . The second condition holds trivially:  $e$  is live in  $C$ , so by safety  $M(\bar{e}) = 0$ ;  $e'$  is live in  $C'$ , so by safety  $M'(\bar{e}') = 0$ ; firing  $t$  produces no token on the dead propagation net, hence  $M''(\bar{e}) = 0 = M'(\bar{e})$  and  $M''(\bar{e}') = 0 = M'(\bar{e}')$ .

**L-AndSplit-NC:** Let  $\text{PN}(\text{andSplit}(e, E)) = (P, T, F)$  with  $P = \{e\} \cup E$ ,  $T = \{t_a\}$ , and  $F = \{(e, t_a)\} \cup \{(t_a, e') \mid e' \in E\}$ . Recall the L-AndSplit-NC rule:

$$\frac{}{\text{andSplit}(e.1, E) \xrightarrow{\alpha} \text{andSplit}(e.d, \text{setLive}(E))}$$

where we let  $C = \text{andSplit}(e.1, E)$ ,  $C' = \text{andSplit}(e.d, \text{setLive}(E))$ ,  $\text{PN}(C) = (N, M)$ , and  $\text{PN}(C') = (N, M')$ . For safety, for every  $e.\Sigma$  in  $E$ , the statuses must be wait. By definition,  $M(e) = 1$  and  $M(e') = 0$  for all  $e' \in E$ , while  $M'(e) = 0$  and  $M'(e') = 1$  for all  $e' \in E$ . The transition  $t_a$  has  $\bullet t_a = \{e\}$  and  $t_a\bullet = E$ ; hence it is enabled under  $M$ . Firing  $t_a$  consumes the token from  $e$  and produces one token in each  $e' \in E$ , yielding a marking  $M''$  such that  $M''(e) = 0$  and  $M''(e') = 1$  for all  $e' \in E$ . The first condition holds since  $M''(e) = M'(e)$  and  $M''(e') = M'(e')$  for all  $e' \in E$ . The second condition holds trivially:  $e$  is live in  $C$ , so by safety  $M(\bar{e}) = 0$ ; every  $e' \in E$  is live in  $C'$ , so by safety  $M'(\bar{e}') = 0$ ; firing  $t_a$  produces no token on the dead propagation net, hence  $M''(\bar{e}) = 0 = M'(\bar{e})$  and  $M''(\bar{e}') = 0 = M'(\bar{e}')$  for all  $e' \in E$ .

**L-AndJoin-NC:** Let  $\text{PN}(\text{andJoin}(E, e)) = (P, T, F)$  with  $P = E \cup \{e\}$ ,  $T = \{t_a\}$ , and  $F = \{(e', t_a) \mid e' \in E\} \cup \{(t_a, e)\}$ . Recall the L-AndJoin-NC rule:

$$\frac{\text{isLive}(E)}{\text{andJoin}(E, e.\Sigma) \xrightarrow{\alpha} \text{andJoin}(\text{setDead}(E), e.1)}$$

where we let  $C = \text{andJoin}(E, e.\Sigma)$ ,  $C' = \text{andJoin}(\text{setDead}(E), e.1)$ ,  $\text{PN}(C) = (N, M)$ , and  $\text{PN}(C') = (N, M')$ . For safety,  $e.\Sigma$  must be wait. By definition,  $M(e') = 1$  for all  $e' \in E$  and  $M(e) = 0$ , while  $M'(e') = 0$  for all  $e' \in E$  and  $M'(e) = 1$ . The transition  $t_a$  has  $\bullet t_a = E$  and  $t_a\bullet = \{e\}$ ; since every place in

$E$  holds a token under  $M$ ,  $t_a$  is enabled. Firing  $t_a$  consumes one token from each  $e' \in E$  and produces one token in  $e$ , yielding a marking  $M''$  such that  $M''(e') = 0$  for all  $e' \in E$  and  $M''(e) = 1$ . The first condition holds since  $M''(e') = M'(e')$  for all  $e' \in E$  and  $M''(e) = M'(e)$ . The second condition holds trivially: every  $e' \in E$  is live in  $C$ , so by safety  $M(\bar{e}') = 0$ ;  $e$  is live in  $C'$ , so by safety  $M'(\bar{e}) = 0$ ; firing  $t_a$  produces no token on the dead propagation net, hence  $M''(\bar{e}') = 0 = M'(\bar{e}')$  for all  $e' \in E$  and  $M''(\bar{e}) = 0 = M'(\bar{e})$ .

**L-XorSplit-NC:** Let  $\text{PN}(\text{xorSplit}(e, E)) = (P, T, F)$  and recall that the encoding introduces, for every  $e' \in E$ , a transition  $t_{e'}$  with  $\bullet t_{e'} = \{e\}$  and  $t_{e'} \bullet = \{e'\} \cup \{\bar{e}' \mid e'' \in E, e'' \neq e'\}$ . Recall also the L-XorSplit-NC rule:

$$\text{xorSplit}(e.1, \{e_s.\Sigma\} \sqcup E') \xrightarrow{\alpha} \text{xorSplit}(e.d, \{e_s.1\} \sqcup \text{setDead}(E'))$$

where  $C = \text{xorSplit}(e.1, \{e_s.\Sigma\} \sqcup E')$ ,  $C' = \text{xorSplit}(e.d, \{e_s.1\} \sqcup \text{setDead}(E'))$ , with  $E = \{e_s\} \sqcup E'$  the full set of outgoing edges,  $\text{PN}(C) = (N, M)$ , and  $\text{PN}(C') = (N, M')$ . For safety,  $e_s.\Sigma$  must be wait. By definition,  $M(e) = 1$  and  $M(e'') = 0$  for all  $e'' \in E$ , while  $M'(e) = 0$ ,  $M'(e_s) = 1$ ,  $M'(e'') = 0$  and  $M'(\bar{e}'') = 1$  for all  $e'' \in E'$ . Since  $\bullet t_{e''} = \{e\}$  for all  $e'' \in E$ , all transitions are enabled under  $M$ ; following the rule above we choose to fire transition  $t_{e_s}$ , which consumes the token from  $e$  and produces, in a single step, one token in  $e_s$  and one token in  $\bar{e}''$  for every  $e'' \in E'$ . The resulting marking  $M''$  satisfies:  $M''(e) = 0$ ,  $M''(e_s) = 1$ ,  $M''(e'') = 0$  and  $M''(\bar{e}'') = 1$  for all  $e'' \in E'$ . The first condition holds since  $M''(e''') = M'(e''')$  for all  $e''' \in E(C)$ . The second condition holds trivially:  $e_s$  is live in  $C'$ , so by safety  $M'(\bar{e}_s) = 0$ ; firing  $t_{e_s}$  produces no token on  $\bar{e}_s$ , hence  $M''(\bar{e}_s) = 0 = M'(\bar{e}_s)$ . For all  $e'' \in E'$ , the Petri net produces  $M''(\bar{e}'') = 1$ , while  $M'(\bar{e}'') \in \{0, 1\}$  depending on whether  $e''$  is a frontier dotted edge; hence  $M''(\bar{e}'') \geq M'(\bar{e}'')$ . Together, both conditions of the theorem are satisfied, concluding the case.

**L-XorJoin-NC:** Let  $\text{PN}(\text{xorJoin}(E, e)) = (P, T, F)$  and recall that the encoding introduces, for every  $e'' \in E$ , a transition  $t_{e''}$  with  $\bullet t_{e''} = \{e''\}$  and  $t_{e''} \bullet = \{e\}$ . Recall also the L-XorJoin-NC rule:

$$\text{xorJoin}(\{e_f.1\} \sqcup E', e.\Sigma) \xrightarrow{\alpha} \text{xorJoin}(\{e_f.d\} \sqcup \text{setDead}(E'), e.1)$$

where  $C = \text{xorJoin}(\{e_f.1\} \sqcup E', e.\Sigma)$ ,  $C' = \text{xorJoin}(\{e_f.d\} \sqcup \text{setDead}(E'), e.1)$ , with  $E = \{e_f\} \sqcup E'$  the full set of incoming edges,  $\text{PN}(C) = (N, M)$ , and  $\text{PN}(C') = (N, M')$ . For safety,  $e.\Sigma$  must be wait. By definition of the encoding,  $M(e_f) = 1$ ,  $M(e'') = 0$  for all  $e'' \in E'$ , and  $M(e) = 0$ , while  $M'(e_f) = 0$ ,  $M'(e'') = 0$  for all  $e'' \in E'$ , and  $M'(e) = 1$ . Since  $\bullet t_{e''} = \{e''\}$  for all  $e'' \in E$ , exactly  $t_{e_f}$  is enabled under  $M$ ; following the rule above we fire transition  $t_{e_f}$ , which consumes the token from  $e_f$  and produces one token in  $e$ . The resulting marking  $M''$  satisfies:  $M''(e_f) = 0$ ,  $M''(e'') = 0$  for all  $e'' \in E'$ , and  $M''(e) = 1$ . The first condition holds since  $M''(e''') = M'(e''')$  for all  $e''' \in E(C)$ . The second condition holds trivially:  $e$  is live in  $C'$ , so by safety  $M'(\bar{e}) = 0$ ; the XOR-join encoding introduces no token on the dead

propagation net in its postset, hence  $M''(\bar{e}) = M(\bar{e}) = 0 = M'(\bar{e})$ . For all  $e'' \in E'$ , firing  $t_{e'}$  produces no token on the dead propagation net, hence  $M''(\bar{e}'') = M(\bar{e}'')$ ; since markings are non-negative and  $M'(\bar{e}'') = 0$ , we get  $M''(\bar{e}'') \geq M'(\bar{e}'')$ . Together, both conditions of the theorem are satisfied, concluding the case.

**L-OrSplit-NC:** Let  $\text{PN}(\text{orSplit}(e, E))$  and recall that the encoding introduces, for every  $S \in \mathcal{P}_\emptyset(E)$ , a transition  $t_S$  with  $\bullet t_S = \{e\}$  and  $t_S \bullet = \{e' \mid e' \in S\} \cup \{\bar{e}'' \mid e'' \in E \setminus S\}$ . Recall also the L-OrSplit-NC rule:

$$\frac{E_1 \neq \emptyset \quad E = \text{setLive}(E_1) \sqcup \text{setDead}(E_2)}{\text{orSplit}(e.1, E_1 \sqcup E_2) \xrightarrow{(\text{d}:\{e\} \sqcup E_2, 1:E_1)} \text{orSplit}(e.d, E)}$$

where  $C = \text{orSplit}(e.1, E_1 \sqcup E_2)$ ,  $C' = \text{orSplit}(e.d, E)$ ,  $\text{PN}(C) = (N, M)$ , and  $\text{PN}(C') = (N, M')$ . The statuses of every edge  $e'.\Sigma \in E$  are wait. Otherwise, this will go against the safety assumption. Then, by definition, the marking of  $\llbracket C \rrbracket$  is  $M(e) = 1$  and for all  $e' \in E$ , the corresponding places  $e'$ ,  $\bar{e}'$  are empty. Since  $M(e) = 1$ , all transitions  $t_S$ , for all  $S \in \mathcal{P}_\emptyset(E)$ , are enabled; following the rule above we choose to fire transition  $t_{E_1}$ , which consumes the token from  $e$  and produces, in a single step, one token in  $e'$  for every  $e' \in E_1$ , and one token in  $\bar{e}''$  for every  $e'' \in E_2$ . The resulting marking  $M''$  satisfies:  $M''(e) = 0$ ,  $M''(e') = 1$  and  $M''(\bar{e}') = 0$  for all  $e' \in E_1$ ,  $M''(e'') = 0$  and  $M''(\bar{e}'') = 1$  for all  $e'' \in E_2$ . It remains to verify the two conditions of the theorem. For the first condition, the live places match by construction:  $M''(e''') = M'(e''')$  for all  $e''' \in \mathbb{E}(C)$ . For the second condition, the outgoing edges in  $E_2$  may or may not be frontier dead edges in  $\llbracket C' \rrbracket$ ; hence  $\llbracket C' \rrbracket$  may carry no token on  $\bar{e}''$  for some  $e'' \in E_2$ . For every  $e'' \in E_2$ , the Petri net produces  $M''(\bar{e}'') = 1$ , while  $\llbracket C' \rrbracket$  carries at most one token on  $\bar{e}''$ , so  $M''(\bar{e}'') \geq M'(\bar{e}'')$ . For every  $e' \in E_1$ , the Petri net produces  $M''(e') = 0$ ; by safety,  $e'$  being live in  $C'$  implies  $M'(e') = 0$ , hence  $M''(e') \geq M'(e')$  holds trivially. Together, both conditions of the theorem are satisfied, concluding the case.

**L-OrJoin-NC:** Let  $\text{PN}(\text{orJoin}(E, e))$  and recall that the encoding introduces, for every  $S \in \mathcal{P}_\emptyset(E)$ , a transition  $t_S$  with  $\bullet t_S = \{e' \mid e' \in S\} \cup \{\bar{e}'' \mid e'' \in E \setminus S\}$  and  $t_S \bullet = \{e\}$ . Recall also the L-OrJoin-NC rule:

$$\frac{\neg \text{isWait}(E) \quad \{e'.\Sigma \in E \mid \Sigma = 1\} \neq \emptyset}{\text{orJoin}(E, e.\Sigma) \xrightarrow{} \text{orJoin}(\text{setDead}(E), e.1)}$$

where  $C = \text{orJoin}(E, e.\Sigma)$ ,  $C' = \text{orJoin}(\text{setDead}(E), e.1)$ ,  $\text{PN}(C) = (N, M)$ , and  $\text{PN}(C') = (N, M')$ . The status of edge  $e.\Sigma$  is wait. Otherwise, this will go against the safety assumption. Since  $\neg \text{isWait}(E)$ , every branch  $e'' \in E$  is either live or dead, i.e. exactly one of  $e''$  or  $\bar{e}''$  holds a token in the marking  $M$ . Let  $E_1 \subseteq E$  be the set of live edges and  $E_2 = E \setminus E_1$  the set of dead edges, with  $E_1 \neq \emptyset$  by hypothesis. Then, by definition, the marking of  $\llbracket C \rrbracket$  is  $M(e') = 1$  for all  $e' \in E_1$ ,  $M(\bar{e}'') = 1$  for all  $e'' \in E_2$ , and all other places are empty. Since the preset of  $t_{E_1}$  is exactly  $\{e' \mid e' \in E_1\} \cup \{\bar{e}'' \mid e'' \in E_2\}$ ,

the transition  $t_{E_1}$  is enabled under  $M$ ; following the rule above we choose to fire transition  $t_{E_1}$ , which consumes, in a single step, the token from each  $e' \in E_1$  and from each  $\bar{e}'' \in E_2$ , and produces one token in  $e$ . The resulting marking  $M''$  satisfies:  $M''(e) = 1$ ,  $M''(e'') = 0$  and  $M''(\bar{e}'') = 0$  for all  $e'' \in E$ . It remains to verify the two conditions of the theorem. For the first condition, the live places match by construction:  $M''(e''') = M'(e''')$  for all  $e''' \in \mathbb{E}(C)$ . For the second condition, the output edge  $e$  may or may not be a frontier dead edge in  $\llbracket C' \rrbracket$ ; hence  $\llbracket C' \rrbracket$  may carry no token on  $\bar{e}$ . The Petri net produces  $M''(\bar{e}) = 0$ , while  $\llbracket C' \rrbracket$  carries at most one token on  $\bar{e}$ , so  $M''(\bar{e}) \geq M'(\bar{e})$  holds trivially. For all  $e'' \in E$ , the Petri net produces  $M''(\bar{e}'') = 0$ ; by safety,  $e$  being live in  $C'$  implies  $M'(\bar{e}'') = 0$ , hence  $M''(\bar{e}'') \geq M'(\bar{e}'')$  holds trivially. Together, both conditions of the theorem are satisfied, concluding the case.

**D-Task:** Let  $\overline{\text{PN}(\text{task}(e, e'))} = (P, T, F)$  where  $P = \{\bar{e}, \bar{e}'\}$ ,  $T = \{\bar{t}\}$ , and  $F = \{(\bar{e}, \bar{t}), (\bar{t}, \bar{e}')\}$ . Recall the D-Task rule:

$$\overline{\text{task}(e.d, e'.w)} \xrightarrow{\alpha} \overline{\text{task}(e.d, e'.d)}$$

where  $C = \text{task}(e.d, e'.\Sigma)$ ,  $C' = \text{task}(e.d, e'.d)$ ,  $\text{PN}(C) = (N, M)$ , and  $\text{PN}(C') = (N, M')$ . By definition,  $M(\bar{e}) = 1$  and  $M(\bar{e}') = 0$ , while  $M'(\bar{e}) = 0$  and  $M'(\bar{e}') \in \{0, 1\}$  depending on whether  $e'$  is a frontier dotted edge. The transition  $\bar{t}$  has  $\bullet \bar{t} = \{\bar{e}\}$ ; hence it is enabled under  $M$ ; following the rule above we fire transition  $\bar{t}$ , which consumes one token from  $\bar{e}$  and produces one token in  $\bar{e}'$ . The resulting marking  $M''$  satisfies:  $M''(\bar{e}) = 0$  and  $M''(\bar{e}') = 1$ . It remains to verify the two conditions of the theorem. For the first condition, no live places are involved in this rule, so  $M''(e''') = M'(e''') = 0$  for all  $e''' \in \mathbb{E}(C)$ . For the second condition, the Petri net produces  $M''(\bar{e}') = 1$ , while  $M'(\bar{e}') \in \{0, 1\}$ , hence  $M''(\bar{e}') \geq M'(\bar{e}')$ . Together, both conditions of the theorem are satisfied, concluding the case.

**D-AndSplit, D-XorSplit, D-OrSplit:** Let  $\overline{\text{PN}(\text{Split}(e, E))} = (P, T, F)$ , where  $P = \{\bar{e}\} \cup \{\bar{e}' \mid e' \in E\}$ ,  $T = \{\bar{t}_a\}$ , and  $F = \{(\bar{e}, \bar{t}_a)\} \cup \{(\bar{t}_a, \bar{e}') \mid e' \in E\}$ . Recall the dead-propagation rule:

$$\overline{\neg \text{isDead}(E) \quad \{e'.\Sigma \in E \mid \Sigma = 1\} = \emptyset} \xrightarrow{\alpha} \overline{\text{Split}(e.d, E) \xrightarrow{\alpha} \text{Split}(e.d, \text{setDead}(E))}$$

where  $\text{Split}$  can be an AND-split, XOR-split, and OR-split  $C = \text{Split}(e.d, E)$ ,  $C' = \text{Split}(e.d, \text{setDead}(E))$ ,  $\text{PN}(C) = (N, M)$ , and  $\text{PN}(C') = (N, M')$ . Since the three rules are identical, we give a single proof. By definition of the dead-propagation net,  $M(\bar{e}) = 1$  and  $M(\bar{e}') = 0$  for all  $e' \in E$ , while  $M'(\bar{e}') \in \{0, 1\}$  depending on whether  $e'$  is a frontier dotted edge. The transition  $\bar{t}_a$  has  $\bullet \bar{t}_a = \{\bar{e}\}$ ; hence it is enabled under  $M$ ; following the rule above we fire transition  $\bar{t}_a$ , which consumes one token from  $\bar{e}$  and produces one token in every  $\bar{e}'$ , for  $e' \in E$ . The resulting marking  $M''$  satisfies  $M''(\bar{e}) = 0$  and  $M''(\bar{e}') = 1$  for all  $e' \in E$ . It remains to verify the two conditions of the theorem. For the first condition, no live places are involved in this

rule. For the second condition, the Petri net produces  $M''(\bar{e}') = 1$ , while  $M'(\bar{e}') \in \{0, 1\}$ , hence  $M''(\bar{e}') \geq M'(\bar{e}')$  for all  $e' \in E$ . Together, both conditions of the theorem are satisfied, concluding the case.

**D-AndJoin, D-XorJoin, D-OrJoin:** Let  $\overline{\text{PN}(\text{Join}(E, e))} = (P, T, F)$ , where  $P = \{\bar{e}' \mid e' \in E\} \cup \{\bar{e}\}$ ,  $T = \{\bar{t}_a\}$ , and  $F = \{(e', \bar{t}_a) \mid e' \in E\} \cup \{(\bar{t}_a, \bar{e})\}$ . Recall the dead-propagation rule:

$$\frac{\text{isDead}(E)}{\text{Join}(E, e.w) \xrightarrow{\alpha} \text{Join}(E, e.d)}$$

where Join can be an AND-join, XOR-join, and OR-join  $C = \text{Join}(E, e.w)$ ,  $C' = \text{Join}(E, e.d)$ ,  $\text{PN}(C) = (N, M)$ , and  $\text{PN}(C') = (N, M')$ . Since the three rules are identical, we give a single proof. By definition of the dead-propagation net,  $M(\bar{e}') = 1$  for all  $e' \in E$ , while  $M(\bar{e}) = 0$ . Moreover,  $M'(\bar{e}) \in \{0, 1\}$  depending on whether  $e$  is a frontier dotted edge. The transition  $\bar{t}_a$  has  $\bullet\bar{t}_a = \{\bar{e}' \mid e' \in E\}$ ; hence it is enabled under  $M$ ; following the rule above we fire transition  $\bar{t}_a$ , which consumes one token from each  $\bar{e}'$  and produces one token in  $\bar{e}$ . The resulting marking  $M''$  satisfies  $M''(\bar{e}') = 0$  for all  $e' \in E$  and  $M''(\bar{e}) = 1$ . It remains to verify the two conditions of the theorem. For the first condition, no live places are involved in this rule. For the second condition, the Petri net produces  $M''(\bar{e}) = 1$ , while  $M'(\bar{e}) \in \{0, 1\}$ , hence  $M''(\bar{e}) \geq M'(\bar{e})$ . Together, both conditions of the theorem are satisfied, concluding the case.

**M-StatusUpd<sub>1</sub>:** The applied rule is

$$\frac{C_1 \xrightarrow{\alpha} C'_1}{C_1 \mid C_2 \xrightarrow{\alpha} C'_1 \mid C_2 \star \alpha}$$

where  $C = C_1 \mid C_2$ ,  $C' = C'_1 \mid C_2 \star \alpha$ ,  $\llbracket C \rrbracket = (N, M)$ , and  $\llbracket C' \rrbracket = (N, M')$ . Since  $C$  is reachable, so is its sub-term  $C_1$ ; moreover, the derivation of  $C_1 \xrightarrow{\alpha} C'_1$  has strictly smaller height than that of  $C \xrightarrow{\alpha} C'$ , so the inductive hypothesis applies to it. Let  $\llbracket C_1 \rrbracket = (N_1, M_1)$  and  $\llbracket C'_1 \rrbracket = (N_1, M'_1)$ . By inductive hypothesis, there exists a firing  $(N_1, M_1) \rightarrow (N_1, M''_1)$  such that  $M''_1(e) = M'_1(e)$  for all  $e \in \mathbb{E}(C_1)$  and  $M''_1(\bar{e}) \geq M'_1(\bar{e})$  for all  $e \in \phi(C_1)$ . By Lemma 3, applied to this firing, there exists a firing  $(N, M) \rightarrow (N, M'')$  such that  $M''(e) = M'(e)$  for all  $e \in \mathbb{E}(C)$  and  $M''(\bar{e}) \geq M'(\bar{e})$  for all  $e \in \phi(C)$ , which are exactly the two conditions required by the theorem, concluding the case. The case **M-StatusUpd<sub>2</sub>** is symmetric, obtained by exchanging the roles of  $C_1$  and  $C_2$  throughout (and applying the symmetric counterpart of Lemma 3).  $\square$

## D Completeness

**Theorem 2 (Completeness).** *Let  $C$  be a reachable marked BPMN term, and  $\llbracket C \rrbracket = (N, M)$  its encoding. If  $(N, M) \rightarrow (N, M'')$ , then there exists a reachable marked BPMN term  $C'$  such that  $C \xrightarrow{\alpha} C'$  and  $(N, M') = \llbracket C' \rrbracket$ , for all  $e \in \mathbb{E}(C)$ ,  $M'(e) = M''(e)$  and for all  $e \in \phi(C)$ ,  $M''(\bar{e}) \geq M'(\bar{e})$ .*

*Proof.* Recall that we only consider well-formed, safe and reachable marked BPMN terms. Safety is exploited to ensure that in case of live token propagation, outgoing edges initially have wait status. We consider here first the cases in which incoming edges are live; the case in which edges are dead, triggering the dead propagation transition, is discussed at the bottom. By structural induction on the BPMN terms  $C$ :

**Start:** Let  $\text{PN}(\text{start}(e)) = (P, T, F)$  with  $P = \{p_s, e\}$ ,  $T = \{t\}$ , and  $F = \{(p_s, t), (t, e)\}$ . Recall the L-Start-NC rule:  $\text{start}(e.w) \xrightarrow{\alpha} \text{start}(e.1)$  where  $C = \text{start}(e.w)$ ,  $C' = \text{start}(e.1)$ ,  $\text{PN}(C) = (N, M)$ , and  $\text{PN}(C') = (N, M')$ . The only transition is  $t$ , so the single firing step  $(N, M) \rightarrow (N, M'')$  must be the firing of  $t$ . By definition,  $M(p_s) = 1$  and  $M(e) = 0$ , so  $t$  is enabled. Firing  $t$  yields  $M''$  with  $M''(p_s) = 0$  and  $M''(e) = 1$ . By rule L-Start-NC we have  $C \xrightarrow{\alpha} C'$ , and by definition  $M'(e) = 1 = M''(e)$ , so the first condition holds. The second condition holds because the  $\text{start}(e)$  introduces no complementary places by definition of  $\phi$ , concluding the case.

**End:** Let  $\text{PN}(\text{end}(e.\Sigma)) = (P, T, F)$  with  $P = \{e\}$ ,  $T = \{t_e\}$ , and  $F = \{(e, t_e)\}$ , so  $\bullet t_e = \{e\}$  and  $t_e \bullet = \emptyset$ . The only transition in the net is  $t_e$ , so the single firing step  $(N, M) \rightarrow (N, M'')$  must be the firing of  $t_e$ . Since  $\bullet t_e = \{e\}$ , the transition  $t_e$  is enabled under  $M$  iff  $M(e) \geq 1$ . The marking encoding gives  $M(e) = 1$  with  $\Sigma = 1$ , which is the only reachable case for an end event before it has fired. Firing  $t_e$  yields  $M''$  with  $M''(e) = M(e) - 1 = 0$ . Setting  $C' = \text{end}(e.d)$ , by rule L-End-NC we have  $C \xrightarrow{\alpha} C'$ , and  $\text{PN}(C') = (N, M')$ . By definition,  $M'(e) = 0 = M''(e)$ , so the first condition holds. The second condition holds because the end event's transition  $t_e$  has empty postset, so no dead propagation place is affected by this firing, concluding the case.

**Task:** Let  $\text{PN}(\text{task}(e.\Sigma, e'.\Sigma')) = (P, T, F)$  with  $P = \{e, e'\}$ ,  $T = \{t_t\}$ , and  $F = \{(e, t_t), (t_t, e')\}$ , so  $\bullet t_t = \{e\}$  and  $t_t \bullet = \{e'\}$ . Since  $\Sigma = 1$ , the transition  $t_t$  is the only one enabled under  $M$ , so the single firing step  $(N, M) \rightarrow (N, M'')$  must be the firing of  $t_t$ . The marking encoding gives  $M(e) = 1$ , which is the only reachable case for a task before it has fired. Firing  $t_t$  yields  $M''$  with  $M''(e) = M(e) - 1 = 0$  and  $M''(e') = M(e') + 1 = 1$ . Setting  $C' = \text{task}(e.d, e'.1)$ , by rule L-Task-NC we have  $C \xrightarrow{\alpha} C'$ , and  $\text{PN}(C') = (N, M')$ . By definition,  $M'(e) = 0 = M''(e)$  and  $M'(e') = 1 = M''(e')$ , so the first condition holds. The second condition holds because the firing  $t_t$  affects no dead propagation place, concluding the case.

**AndSplit:** Let  $\text{PN}(\text{andSplit}(e.\Sigma, E)) = (P, T, F)$  with  $P = \{e\} \cup E$ ,  $T = \{t_a\}$ , and  $F = \{(e, t_a)\} \cup \{(t_a, e') \mid e' \in E\}$ , so  $\bullet t_a = \{e\}$  and  $t_a \bullet = E$ . Since  $\Sigma = 1$ , the transition  $t_a$  is the only one enabled under  $M$ , so the single firing step  $(N, M) \rightarrow (N, M'')$  must be the firing of  $t_a$ . The marking encoding gives  $M(e) = 1$ , which is the only reachable case for an and-split before it has fired. Firing  $t_a$  yields  $M''$  with  $M''(e) = M(e) - 1 = 0$  and  $M''(e') = M(e') + 1 = 1$  for all  $e' \in E$ . Setting  $C' = \text{andSplit}(e.d, \text{setLive}(E))$ , by rule L-AndSplit-NC we have  $C \xrightarrow{\alpha} C'$ , and  $\text{PN}(C') = (N, M')$ . By definition,  $M'(e) = 0 = M''(e)$  and  $M'(e') = 1 = M''(e')$  for all  $e' \in E$ , so the first condition holds. The

second condition holds vacuously: firing  $t_a$  affects no dead propagation place, concluding the case.

**AndJoin:** Let  $\text{PN}(\text{andJoin}(E, e, \Sigma)) = (P, T, F)$  with  $P = E \cup \{e\}$ ,  $T = \{t_a\}$ , and  $F = \{(e'', t_a) \mid e'' \in E\} \cup \{(t_a, e)\}$ , so  $\bullet t_a = E$  and  $t_a \bullet = \{e\}$ . Since  $\text{isLive}(E)$ , the transition  $t_a$  is the only one enabled under  $M$ , so the single firing step  $(N, M) \rightarrow (N, M'')$  must be the firing of  $t_a$ . The marking encoding gives  $M(e'') = 1$  for all  $e'' \in E$ , which is the only reachable case for an and-join before it has fired. Firing  $t_a$  yields  $M''$  with  $M''(e'') = M(e'') - 1 = 0$  for all  $e'' \in E$  and  $M''(e) = M(e) + 1 = 1$ . Setting  $C' = \text{andJoin}(\text{setDead}(E), e, 1)$ , by rule **L-AndJoin-NC** we have  $C \xrightarrow{\alpha} C'$ , and  $\text{PN}(C') = (N, M')$ . By definition,  $M'(e'') = 0 = M''(e'')$  for all  $e'' \in E$  and  $M'(e) = 1 = M''(e)$ , so the first condition holds. The second condition holds vacuously: firing  $t_a$  affects no dead propagation place, concluding the case.

**XorSplit:** Let  $\text{PN}(\text{xorSplit}(e, \Sigma, E)) = (P, T, F)$  with  $P = \{e\} \cup E \cup \{\bar{e}'' \mid e'' \in E\}$ ,  $T = \{t_{e''} \mid e'' \in E\}$ , and  $F = \{(e, t_{e''}), (t_{e''}, e'')\}_{e'' \in E} \cup \{(t_{e''}, \bar{e}''') \mid e'', e''' \in E, e''' \neq e''\}$ , so  $\bullet t_{e''} = \{e\}$  and  $t_{e''} \bullet = \{e''\} \cup \{\bar{e}''' \mid e''' \in E \setminus \{e''\}\}$  for each  $e'' \in E$ . The marking encoding gives  $M(e) = 1$  and, since  $e$  is live,  $M(e'') = 0$  for all  $e'' \in E$ , which is the only reachable case for an xor-split before it has fired; hence all transitions  $t_{e''}$  are enabled under  $M$ . The single firing step  $(N, M) \rightarrow (N, M'')$  is the firing of some  $t_{e'}$  for a chosen  $e' \in E$ . Firing  $t_{e'}$  yields  $M''$  with  $M''(e) = 0$ ,  $M''(e') = 1$ ,  $M''(e'') = 0$  for all  $e'' \in E \setminus \{e'\}$ , and  $M''(\bar{e}'') = 1$  for all  $e'' \in E \setminus \{e'\}$ . Setting  $C' = \text{xorSplit}(e, \bar{e}', 1) \sqcup \text{setDead}(E \setminus \{e'\})$ , by rule **L-XorSplit-NC** we have  $C \xrightarrow{\alpha} C'$ , and  $\text{PN}(C') = (N, M')$ . By definition,  $M'(e) = 0 = M''(e)$  and  $M'(e') = 1 = M''(e')$ , so the first condition holds. For the second condition,  $M''(\bar{e}') = 0$ , and since  $e'$  is live in  $C'$ , by safety  $M'(\bar{e}') = 0$ , so  $M''(\bar{e}') \geq M'(\bar{e}')$  holds trivially. For all  $e'' \in E \setminus \{e'\}$ , the Petri net produces  $M''(\bar{e}'') = 1$ , while  $M'(\bar{e}'') \in \{0, 1\}$  depending on whether  $e''$  is a frontier dead edge, hence  $M''(\bar{e}'') \geq M'(\bar{e}'')$ . Together, both conditions of the theorem are satisfied, concluding the case.

**XorJoin:** Let  $\text{PN}(\text{xorJoin}(E, e, \Sigma)) = (P, T, F)$  with  $P = E \cup \{e\}$ ,  $T = \{t_{e''} \mid e'' \in E\}$ , and  $F = \{(e'', t_{e''}), (t_{e''}, e)\}_{e'' \in E}$ , so  $\bullet t_{e''} = \{e''\}$  and  $t_{e''} \bullet = \{e\}$  for each  $e'' \in E$ . The marking encoding gives  $M(e') = 1$  for the live edge  $e'$  and  $M(e'') = 0$  for all  $e'' \in E \setminus \{e'\}$ . Since  $\bullet t_{e''} = \{e''\}$  for all  $e'' \in E$ , only  $t_{e'}$  is enabled under  $M$ , so the single firing step  $(N, M) \rightarrow (N, M'')$  must be the firing of  $t_{e'}$ . Firing  $t_{e'}$  yields  $M''$  with  $M''(e') = 0$ ,  $M''(e'') = 0$  for all  $e'' \in E \setminus \{e'\}$ , and  $M''(e) = 1$ . Setting  $C' = \text{xorJoin}(\{e', \bar{e}'\} \sqcup \text{setDead}(E \setminus \{e'\}), e, 1)$ , by rule **L-XorJoin-NC** we have  $C \xrightarrow{\alpha} C'$ , and  $\text{PN}(C') = (N, M')$ . By definition,  $M'(e') = 0 = M''(e')$ ,  $M'(e'') = 0 = M''(e'')$  for all  $e'' \in E \setminus \{e'\}$ , and  $M'(e) = 1 = M''(e)$ , so the first condition holds. The second condition holds vacuously: firing  $t_{e'}$  affects no dead propagation place, concluding the case.

**OrSplit:** Let  $\text{PN}(\text{orSplit}(e, \Sigma, E)) = (P, T, F)$  and recall that the encoding introduces, for every  $S \in \mathcal{P}_0(E)$ , a transition  $t_S$  with  $\bullet t_S = \{e\}$  and  $t_S \bullet = \{e' \mid e' \in S\} \cup \{\bar{e}'' \mid e'' \in E \setminus S\}$ . The marking encoding gives  $M(e) = 1$ , and since  $e$  is live,  $M(e') = 0$  and  $M(\bar{e}') = 0$  for all  $e' \in E$  which is the only reachable case for an OR-split before it has fired. Since  $M(e) = 1$ ,

all transitions  $t_S$ , for  $S \in \mathcal{P}_\emptyset(E)$ , are enabled under  $M$ ; the single firing step  $(N, M) \rightarrow (N, M'')$  is the firing of some transition  $t_S$ , for a chosen  $S \in \mathcal{P}_\emptyset(E)$ . Set  $E_1 := S$  and  $E_2 := E \setminus S$ ; since  $S \in \mathcal{P}_\emptyset(E)$ ,  $E_1 \neq \emptyset$ . Firing  $t_{E_1}$  consumes the token from  $e$  and produces, in a single step, one token in  $e'$  for every  $e' \in E_1$ , and one token in  $\bar{e}''$  for every  $e'' \in E_2$ . The resulting marking  $M''$  satisfies:  $M''(e) = 0$ ,  $M''(e') = 1$  and  $M''(\bar{e}') = 0$  for all  $e' \in E_1$ ,  $M''(e'') = 0$  and  $M''(\bar{e}'') = 1$  for all  $e'' \in E_2$ . Recall the L-OrSplit-NC rule:

$$\frac{E_1 \neq \emptyset \quad E = \text{setLive}(E_1) \sqcup \text{setDead}(E_2)}{\text{orSplit}(e.1, E_1 \sqcup E_2) \xrightarrow{(\text{d}:\{e\} \sqcup E_2, 1:E_1)} \text{orSplit}(e.d, E)}$$

Setting  $C' = \text{orSplit}(e.d, \text{setLive}(E_1) \sqcup \text{setDead}(E_2))$ , by rule L-OrSplit-NC we have  $C \xrightarrow{\alpha} C'$ , and  $\text{PN}(C') = (N, M')$ . By definition,  $M'(e) = 0 = M''(e)$  and  $M'(e') = 1 = M''(e')$  for all  $e' \in E_1$ , so the first condition holds. For the second condition, for all  $e' \in E_1$ ,  $M''(\bar{e}') = 0$ , and since  $e'$  is live in  $C'$ , by safety  $M'(\bar{e}') = 0$ , so  $M''(\bar{e}') \geq M'(\bar{e}')$  holds trivially. For all  $e'' \in E_2$ , the Petri net produces  $M''(\bar{e}'') = 1$ , while  $M'(\bar{e}'') \in \{0, 1\}$  depending on whether  $e''$  is a frontier dead edge, hence  $M''(\bar{e}'') \geq M'(\bar{e}'')$ . Together, both conditions of the theorem are satisfied, concluding the case.

**OrJoin:** Let  $\text{PN}(\text{orJoin}(E, e.\Sigma)) = (P, T, F)$  and recall that the encoding introduces, for every  $S \in \mathcal{P}_\emptyset(E)$ , a transition  $t_S$  with  $\bullet t_S = \{e' \mid e' \in S\} \cup \{\bar{e}'' \mid e'' \in E \setminus S\}$  and  $t_S \bullet = \{e\}$ . By definition, the marking encoding gives, for each  $e'' \in E$ , exactly one of  $M(e'') = 1$  or  $M(\bar{e}'') = 1$ , corresponding to  $\neg \text{isWait}(E)$ , which is the only reachable case for an OR-join before it has fired. Since  $\bullet t_S$  for  $S \in \mathcal{P}_\emptyset(E)$  requires a token in  $e'$  for every  $e' \in S$  and in  $\bar{e}''$  for every  $e'' \in E \setminus S$ , exactly one transition  $t_S$  is enabled under  $M$ , namely for  $S$  equal to the set of currently live edges; the single firing step  $(N, M) \rightarrow (N, M'')$  must be the firing of this transition  $t_S$ . Set  $E_1 := S$  and  $E_2 := E \setminus S$ . We consider here the case  $E_1 \neq \emptyset$ , as required by rule L-OrJoin-NC below; the case  $E_1 = \emptyset$  corresponds to dead-status propagation and is treated in Section D. By definition,  $M(e') = 1$  for all  $e' \in E_1$ ,  $M(\bar{e}'') = 1$  for all  $e'' \in E_2$ , and all other places are empty. Recall the L-OrJoin-NC rule:

$$\frac{\neg \text{isWait}(E) \quad \{e' . \Sigma \in E \mid \Sigma = 1\} \neq \emptyset}{\text{orJoin}(E, e.\Sigma) \xrightarrow{} \text{orJoin}(\text{setDead}(E), e.1)}$$

Firing  $t_{E_1}$  consumes, in a single step, the token from each  $e' \in E_1$  and from each  $\bar{e}'' \in E_2$ , and produces one token in  $e$ . The resulting marking  $M''$  satisfies:  $M''(e) = 1$ ,  $M''(e'') = 0$  and  $M''(\bar{e}'') = 0$  for all  $e'' \in E$ . Setting  $C' = \text{orJoin}(\text{setDead}(E), e.1)$ , by rule L-OrJoin-NC we have  $C \xrightarrow{\alpha} C'$ , and  $\text{PN}(C') = (N, M')$ . By definition,  $M'(e) = 1 = M''(e)$ ,  $M'(e'') = 0 = M''(e'')$  for all  $e'' \in E$ , so the first condition holds. For the second condition, for all  $e'' \in E$ , the Petri net produces  $M''(\bar{e}'') = 0$ , and since  $e$  is live in  $C'$ , by safety  $M'(\bar{e}'') = 0$ , so  $M''(\bar{e}'') \geq M'(\bar{e}'')$  holds trivially. Together, both conditions of the theorem are satisfied, concluding the case.

**Composition:** The component is  $C = C_1 \mid C_2$ , with  $\llbracket C \rrbracket = \llbracket C_1 \rrbracket \oplus \llbracket C_2 \rrbracket$ , so  $N = N_1 \oplus N_2$  and, by definition of  $\llbracket \cdot \rrbracket$  on compositions,  $M(p) = M_1(p)$  for all  $p \in P_1$  and  $M(p) = M_2(p)$  for all  $p \in P_2$ , where  $(N_1, M_1) = \llbracket C_1 \rrbracket$  and  $(N_2, M_2) = \llbracket C_2 \rrbracket$ . The transition  $t$  fired in the single step  $(N, M) \rightarrow (N, M'')$  belongs to either  $T_1$  or  $T_2$ . Without loss of generality, assume  $t \in T_1$  (the other case is symmetric, using rule **M-StatusUpd<sub>2</sub>**). Since  $\bullet t, t\bullet \subseteq P_1$ , firing  $t$  leaves every place in  $P_2$  unchanged, i.e.  $M''(p) = M_2(p)$  for all  $p \in P_2$ , and restricts to a firing  $(N_1, M_1) \rightarrow (N_1, M'_1)$  with  $M'_1 = M''|_{P_1}$ . By inductive hypothesis applied to  $C_1$ , there exists a reachable marked BPMN term  $C'_1$  such that  $C_1 \xrightarrow{\alpha} C'_1$  and  $\llbracket C'_1 \rrbracket = (N_1, M'_1)$ , with  $M'_1(e) = M''(e)$  for all  $e \in \mathbb{E}(C_1)$  and  $M'_1(\bar{e}) \geq M''(\bar{e})$  for all  $e \in \phi(C_1)$ . Recall the **M-StatusUpd<sub>1</sub>** rule:

$$\frac{C_1 \xrightarrow{\alpha} C'_1}{C_1 \mid C_2 \xrightarrow{\alpha} C'_1 \mid C_2 \star \alpha}$$

Setting  $C' = C'_1 \mid C_2 \star \alpha$ , we thus have  $C \xrightarrow{\alpha} C'$ . The firing  $(N_1, M_1) \rightarrow (N_1, M'_1)$  together with  $C_1 \xrightarrow{\alpha} C'_1$  satisfy the hypotheses of Lemma 3, which yields a firing  $(N, M) \rightarrow (N, M''')$  such that  $M'''(e) = M'(e)$  for all  $e \in \mathbb{E}(C)$  and  $M'''(\bar{e}) \geq M'(\bar{e})$  for all  $e \in \phi(C)$ , where  $(N, M') = \llbracket C' \rrbracket$ . Since firing a transition from a given marking is deterministic (Definition 2), and both  $M''$  and  $M'''$  arise from firing the same transition  $t$  from the same marking  $M$ , we have  $M''' = M''$ . Hence  $M''(e) = M'(e)$  for all  $e \in \mathbb{E}(C)$ , so the first condition holds, and  $M''(\bar{e}) \geq M'(\bar{e})$  for all  $e \in \phi(C)$ , so the second condition holds, concluding the case.

**Dead Propagation Task:** When  $e$  is in  $\phi(C)$ , by definition, the full encoding  $\text{task}(e, e')$  also includes the dead propagation net  $\text{PN}(\text{task}(e, e'))$  with  $P = \{\bar{e}, \bar{e}'\}$ ,  $T = \{\bar{t}_t\}$ , and  $F = \{(\bar{e}, \bar{t}_t), (\bar{t}_t, \bar{e}')\}$ , so  $\bullet \bar{t}_t = \{\bar{e}\}$  and  $\bar{t}_t \bullet = \{\bar{e}'\}$ . Recall the **D-Task** rule:

$$\text{task}(e.d, e'.w) \xrightarrow{d:e'} \text{task}(e.d, e'.d)$$

The marking encoding gives  $M(\bar{e}) = 1$  with  $\Sigma = d$ , which is the only reachable case triggering dead propagation; hence  $\bar{t}_t$  is the only transition enabled under  $M$ , so the single firing step  $(N, M) \rightarrow (N, M'')$  must be the firing of  $\bar{t}_t$ . Firing  $\bar{t}_t$  yields  $M''$  with  $M''(\bar{e}) = 0$  and  $M''(\bar{e}') = 1$ . Setting  $C' = \text{task}(e.d, e'.d)$ , by rule **D-Task** we have  $C \xrightarrow{\alpha} C'$ , and  $\text{PN}(C') = (N, M')$ . The first condition holds trivially: no live place is involved in this rule, so  $M''(e''') = M'(e''') = 0$  for all  $e''' \in \mathbb{E}(C)$ . For the second condition,  $e, e' \in \phi$ ; the Petri net produces  $M''(\bar{e}) = 0$ , and since  $M'(\bar{e}) = 0$  by definition of  $C'$  (the incoming dead status has already been propagated),  $M''(\bar{e}) \geq M'(\bar{e})$  holds trivially. For  $e'$ , the Petri net produces  $M''(\bar{e}') = 1$ , while  $M'(\bar{e}') \in \{0, 1\}$  depending on whether  $e'$  is a frontier dead edge, hence  $M''(\bar{e}') \geq M'(\bar{e}')$ . Together, both conditions of the theorem are satisfied, concluding the case.

**Dead Propagation Split Gateways** The dead-propagation rules for all the split gateways have the same premises and effects. Therefore, it is sufficient to

prove the result for the XOR-split gateway, as the proofs for the AND-split and OR-split gateways are analogous. Consider the rule

$$\frac{\neg \text{isDead}(E), \quad \{e'.\Sigma \in E \mid \Sigma = 1\} = \emptyset}{\text{xorSplit}(e.d, E) \xrightarrow{\text{d}:E} \text{xorSplit}(e.d, \text{setDead}(E))}$$

with  $\neg \text{isDead}(E)$  and  $\{e'.\Sigma \in E \mid \Sigma = 1\} = \emptyset$ . By definition of the dead propagation encoding,  $\overline{\text{PN}(\text{xorSplit}(e, E))}$  is an AND-split net on overlined edges, with  $P = \{\bar{e}\} \cup \{\bar{e}' \mid e' \in E\}$ ,  $T = \{t_a\}$ , and  $F = \{(\bar{e}, t_a)\} \cup \{(t_a, \bar{e}') \mid e' \in E\}$ , so  $\bullet t_a = \{\bar{e}\}$  and  $t_a \bullet = \{\bar{e}' \mid e' \in E\}$ . Since  $e$  is dead, the marking encoding gives  $M(\bar{e}) = 1$ , which is the only reachable case triggering dead propagation; hence  $t_a$  is the only transition enabled under  $M$ , so the single firing step  $(N, M) \rightarrow (N, M'')$  must be the firing of  $t_a$ . Firing  $t_a$  yields  $M''$  with  $M''(\bar{e}) = 0$  and  $M''(\bar{e}') = 1$  for all  $e' \in E$ . Setting  $C' = \text{xorSplit}(e.d, \text{setDead}(E))$ , by rule **D-XorSplit** we have  $C \xrightarrow{\alpha} C'$ , and  $\text{PN}(C') = (N, M')$ . The first condition holds trivially: no live place is involved in this rule, so  $M''(e''') = M'(e''') = 0$  for all  $e''' \in \mathbb{E}(C)$ . For the second condition, the Petri net produces  $M''(\bar{e}) = 0$ , and since  $M'(\bar{e}) = 0$  by definition of  $C'$  (the incoming dead status of  $e$  has already been propagated), we have  $M''(\bar{e}) \geq M'(\bar{e})$  trivially. For all  $e' \in E$ , the Petri net produces  $M''(\bar{e}') = 1$ , while  $M'(\bar{e}') \in \{0, 1\}$  depending on whether  $e'$  is a frontier dead edge, hence  $M''(\bar{e}') \geq M'(\bar{e}')$ . Together, both conditions of the theorem are satisfied, concluding the case.

**Dead Propagation Join Gateways** The dead-propagation rules for all join gateways have identical premises and effects. Therefore, it is sufficient to prove the result for the XOR-join gateway, as the proofs for the AND-join and OR-join gateways are analogous. Consider the rule

$$\frac{\text{isDead}(E)}{\text{xorJoin}(E, e.w) \xrightarrow{\text{d}:e} \text{xorJoin}(E, e.d)}$$

with  $\text{isDead}(E)$ . By definition,  $\overline{\text{PN}(\text{xorJoin}(E, e))}$  is an AND-join net on overlined edges, with  $P = \{\bar{e}'' \mid e'' \in E\} \cup \{\bar{e}\}$ ,  $T = \{t_a\}$ , and  $F = \{(\bar{e}'', t_a) \mid e'' \in E\} \cup \{(t_a, \bar{e})\}$ , so  $\bullet t_a = \{\bar{e}'' \mid e'' \in E\}$  and  $t_a \bullet = \{\bar{e}\}$ . Since  $\text{isDead}(E)$ , the marking encoding gives  $M(\bar{e}'') = 1$  for all  $e'' \in E$ , which is the only reachable case triggering dead propagation; hence  $t_a$  is the only transition enabled under  $M$ , so the single firing step  $(N, M) \rightarrow (N, M'')$  must be the firing of  $t_a$ . Firing  $t_a$  yields  $M''$  with  $M''(\bar{e}'') = 0$  for all  $e'' \in E$  and  $M''(\bar{e}) = 1$ . Setting  $C' = \text{xorJoin}(E, e.d)$ , by rule **D-XorJoin** we have  $C \xrightarrow{\alpha} C'$ , and  $\text{PN}(C') = (N, M')$ . The first condition holds trivially: no live place is involved in this rule, so  $M''(e''') = M'(e''') = 0$  for all  $e''' \in \mathbb{E}(C)$ . For the second condition, for all  $e'' \in E$ , the Petri net produces  $M''(\bar{e}'') = 0$ , and since  $M'(\bar{e}'') = 0$  by definition of  $C'$  (the incoming dead status of  $e''$  has already been propagated), we have  $M''(\bar{e}'') \geq M'(\bar{e}'')$  trivially. For  $e$ , the Petri net produces  $M''(\bar{e}) = 1$ , while  $M'(\bar{e}) \in \{0, 1\}$  depending on whether

$e$  is a frontier dead edge, hence  $M''(\bar{e}) \geq M'(\bar{e})$ . Together, both conditions of the theorem are satisfied, concluding the case.  $\square$